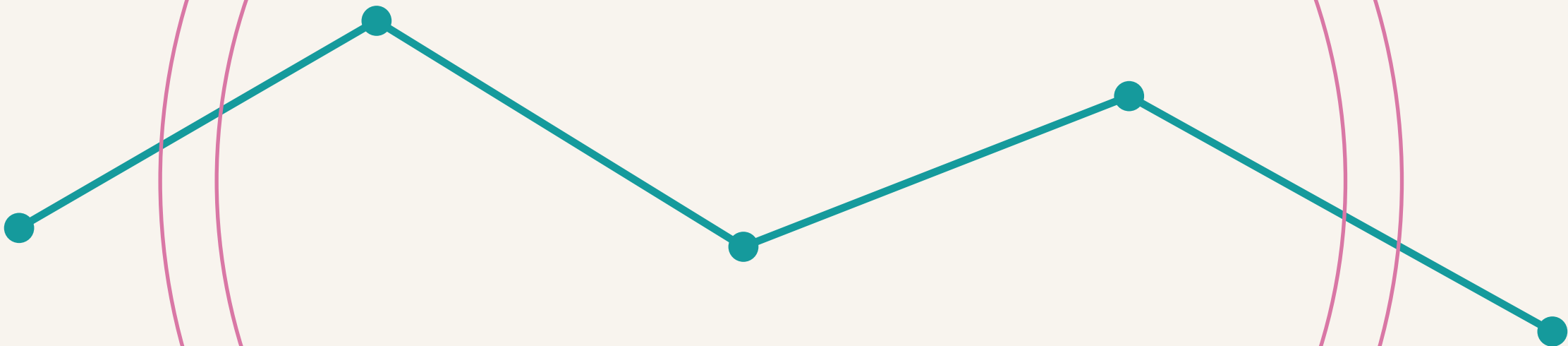


WILDTRACK CONSERVATION ALLIANCE

Wildlife Conservation Database

SQL DATABASE PROJECT REPORT



PREPARED BY

R Hemant Kumar Patra

TECHNOLOGY

MySQL / SQL

DATABASE

Project_2

PROJECT TYPE

Relational database + analytics

SOURCE SCOPE

10,000 records • 4 tables

REPORT

Academic / Portfolio

SQL DATABASE MANAGEMENT & ANALYTICS

A professional case-study report covering schema design, SQL analysis, data integrity, performance, security, and 75 real-world conservation business scenarios.

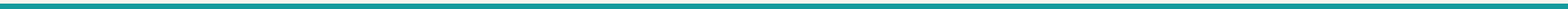


Table of Contents

The report is organized for academic submission, portfolio documentation, GitHub reference, and SQL/Data Analyst interview discussion.

Table of Contents	2
1. Executive Summary	8
2. Project Introduction	8
Project Information	8
Why a Relational Database?	8
3. Business Scenario	8
4. Problem Statement	9
5. Project Objectives	9
6. Scope of the Project	9
7. Database Overview	10
8. Database Architecture	10
9. EER Diagram Explanation	10
10. Database Tables	11
10.1 Reserves	11
10.2 Species	11
10.3 Animals	11
10.4 GPSLogs	11
11. Relationships and Keys	12
12. Data Types and Constraints	12
13. SQL Concepts Used	12
14. Detailed Analysis of the 75 Business Cases	13

Case 1 — Withdraw Researcher Access	13
Case 2 — Flag Animals with Fewer Than 5 Pings	14
Case 3 — Find Reserves with No Collared Animals	14
Case 4 — Count Monitored Species	15
Case 5 — Flag Long Reserve Names	15
Case 6 — Classify Animals by Age	16
Case 7 — Rolling Three-Reading Battery Average	17
Case 8 — Find Animals with No Recent Reports	17
Case 9 — Normalize Tag-Code Capitalization	18
Case 10 — Species Population and Reserve Concentration	19
Case 11 — Find Collars Older Than Two Years	20
Case 12 — Find Lowest Battery Reading	20
Case 13 — Movement by Day of Week	21
Case 14 — Create Reserve Battery Reporting View	21
Case 15 — Retrieve June GPS Logs	22
Case 16 — Running Collaring Count by Species	23
Case 17 — Combine Animal, Species and Reserve Details	24
Case 18 — Prevent Invalid Battery Percentages	24
Case 19 — Elapsed Hours Between First and Latest Log	25
Case 20 — Days Since Collar Fitting	25
Case 21 — Classify Reserve Monitoring Density	26
Case 22 — Average Age of Critically Endangered Animals	27
Case 23 — Classify Battery Health	27
Case 24 — High Average-Speed Reserves in Last Month	28

Case 25 — Species with Fewer Than 10 Individuals	29
Case 26 — Detect Unusually High Individual Speeds	30
Case 27 — Find Reserve of Highest-Speed Animal	30
Case 28 — Index GPS Logs for Animal Lookup	31
Case 29 — Update Collar-Fitting Date	32
Case 30 — Build Animal Display Labels	32
Case 31 — Count Collared Animals by Reserve	33
Case 32 — Find Animals with Fewer Than 3 Monthly Logs	34
Case 33 — Alliance-Wide Average Battery	34
Case 34 — Species–Reserve Coverage Pairs	35
Case 35 — Animals in the Three Largest Reserves	35
Case 36 — Animals with No GPS Pings	36
Case 37 — Total Tracked Animals	37
Case 38 — Add Collar Manufacturer Attribute	37
Case 39 — Prevent Duplicate Tag Registrations	38
Case 40 — Fastest Animal per Reserve	38
Case 41 — Highest Speed Ever Recorded	39
Case 42 — Reserves with More Than 15 Animals	40
Case 43 — Reserves Below 20% Average Battery	40
Case 44 — Consolidated Reserve Health Report	41
Case 45 — Correct Battery Calibration	42
Case 46 — Combined Reserve and Animal Audit	43
Case 47 — Species in Reserves Established Before 2000	43
Case 48 — Find Orphan GPS Records	44

Case 49 — Rank Reserves by Activity Score	45
Case 50 — Remove First-Generation Collar Logs	45
Case 51 — Require Species for Every Animal	46
Case 52 — Compare Reserve Battery with Alliance Average	47
Case 53 — Optimize Combined Reserve/Date Searches	48
Case 54 — Trim Reserve Names	48
Case 55 — Latest Animal Battery Snapshot	49
Case 56 — Reusable Critically Endangered Animal View	50
Case 57 — Count Total GPS Pings	50
Case 58 — Reject Negative Estimated Age	51
Case 59 — Create Three Age Tiers	51
Case 60 — All Logs for the Lowest-Battery Animal	52
Case 61 — Correct Reserve Assignment in Bulk	53
Case 62 — Show the Next Speed Reading	54
Case 63 — Order Animals by Age Within Reserve	54
Case 64 — Reject Unknown Animal IDs in GPS Logs	55
Case 65 — Find Reserves with No GPS Activity in Seven Days	56
Case 66 — Dense-Rank GPS Logs by Speed	56
Case 67 — Quarterly Collaring Activity	57
Case 68 — Show Previous Battery Reading	58
Case 69 — Read-Only Field Researcher Access	58
Case 70 — Latest Coordinates for Every Animal	59
Case 71 — Veterinarian Update Access	60
Case 72 — Animals of the Most-Tracked Species	60

Case 73 — Inspect GPS Log Indexes	61
Case 74 — Classify Movement by Speed	62
Case 75 — Nocturnal GPS Logs	62
15. Query Categories	63
Database Administration & Security	63
Data Integrity & Constraints	63
Aggregation & Grouping	63
Joins	63
Subqueries & CTEs	63
Window Functions	63
Date & Time Analysis	63
String Functions	64
Performance Optimization	64
Views	64
Data Cleaning & Correction	64
16. Data Quality & Validation	64
17. Database Security	64
18. Indexing & Performance Optimization	65
19. Views	65
20. Key Findings & Insights	65
21. Business / Conservation Applications	66
Ranger Operations	66
Collar Maintenance	66

Wildlife Research 66

Conservation Planning 66

Management Reporting 66

Donor Reporting 66

Emergency Monitoring 66

22. Challenges & Solutions 66

23. Project Workflow 66

24. Conclusion 67

25. Future Improvements 67

26. Skills Demonstrated 67

27. Appendix — Important SQL Queries 68

 A. Core Table Creation 68

 B. Important Analytical Patterns 68

 C. Index Definitions 69

 D. Security Examples 69

 E. Source and Validation Notes 70

1. Executive Summary

WildTrack Conservation Alliance is presented as a multi-country wildlife conservation programme that protects endangered species across a network of forest reserves and national parks. Field rangers fit GPS tracking collars to individual animals; the collars return operational measurements including location, speed, temperature and battery health. The supplied scenario emphasizes that the resulting database is central to patrol planning, equipment monitoring, anomaly detection, conservation analysis and reporting. ■filecite■turn0file0■L5-L20■

The supplied implementation uses a MySQL relational database named **Project_2** and organizes the operational model into four connected tables: Reserves, Species, Animals and GPSLogs. The project materials describe approximately 10,000 records across those four tables, with GPS logs forming the majority. ■filecite■turn0file1■L16-L23■

From a data-analyst/database perspective, the project demonstrates how SQL can move from basic record retrieval to operational controls: aggregation, joins, subqueries, CTEs, window functions, date and string functions, constraints, views, indexes, updates, deletion, and user privileges. The 75 business cases provide a practical interview-style test of choosing the right SQL technique for a real requirement.

Source boundary: Project-specific facts in this report are based on the supplied SQL script, scenario, and MySQL Workbench model. Where the source query is incomplete, unsafe, or not executable against the supplied schema, that limitation is explicitly identified rather than silently hidden.

2. Project Introduction

The database models a wildlife tracking operation in which each monitored animal is associated with a species and reserve, while repeated GPS readings are stored against the animal. This structure separates relatively stable master data (reserves and species), operational animal records, and high-volume telemetry data.

The supplied scenario specifically identifies location, speed, temperature, battery health and timestamp as collar-generated information. ■filecite■turn0file0■L5-L18■

Project Information

Item	Value
Project title	WildTrack Conservation Alliance — Wildlife Conservation Database
Project type	SQL Database Design, Data Analysis & Real-World SQL Problem Solving
Prepared by	R Hemant Kumar Patra
Technology	MySQL / SQL
Database	Project_2
Core tables	Reserves, Species, Animals, GPSLogs
Source dataset size	10,000 valid seed records across the four core tables

Why a Relational Database?

A relational model allows the project to avoid repeatedly storing reserve and species descriptions in every GPS record. Instead, keys connect entities and let analysts reconstruct the required business view with joins. The design also creates a natural place for integrity rules: unique identifiers, required fields, valid numeric ranges, and foreign-key relationships.

3. Business Scenario

The scenario describes an operational database used by rangers, biologists, the data team and donor relations. The database is therefore not only an archive: it is a decision-support layer for field activity, equipment maintenance, research, management reporting and funding communication. ■filecite■turn0file0■L10-L23■

Wildlife monitoring

Animal and GPS records let teams reconstruct where monitored individuals are and how frequently they report.

Ranger operations

Counts, recent activity and unusual-speed analyses can help prioritize field attention.

Collar maintenance

Battery readings and reporting gaps provide signals for maintenance or replacement.

Research

Age, species, reserve and movement fields can be combined for population and behavior analysis.

Conservation planning

Species coverage, reserve coverage and conservation status can be summarized for planning.

Reporting

Views and aggregate queries create repeatable outputs for management and donor-facing reporting.

Data quality

Constraints and diagnostic queries can identify invalid values, duplicates, missing relationships and orphan records.

Database administration

The cases also cover users, privileges and performance indexes, reflecting a production-style database environment.

4. Problem Statement

Wildlife conservation teams need a centralized relational database capable of managing reserves, species, individual animals and GPS readings while preserving data integrity. The system must also support operational questions such as inactive collars, low battery levels and unusual movement, and must provide controlled access for different categories of users. The supplied project explicitly frames the database as the center of field operations, analysis and reporting.

■filecite■turn0file0■L10-L18■

5. Project Objectives

1. Design and use a relational wildlife-conservation database.
2. Store reserve, species, animal and GPS tracking information.
3. Establish primary-key and foreign-key relationships.
4. Protect data quality through NOT NULL, UNIQUE and CHECK constraints.
5. Solve operational and analytical requirements using SQL.
6. Analyze GPS activity and collar battery health.
7. Identify inactive, low-reporting or potentially problematic collars.
8. Generate repeatable reporting outputs using views and aggregation.
9. Improve lookup performance with indexes.
10. Demonstrate database security using users and privileges.

6. Scope of the Project

The supplied materials cover four core entities, a 10,000-record seed dataset, 75 business cases, and database-administration examples. The scope includes DDL, DML, DQL, analytical SQL, data-quality controls, views, indexes and access control. Recommendations such as Python analytics, machine learning, dashboards and cloud deployment are treated as future improvements, not existing features.

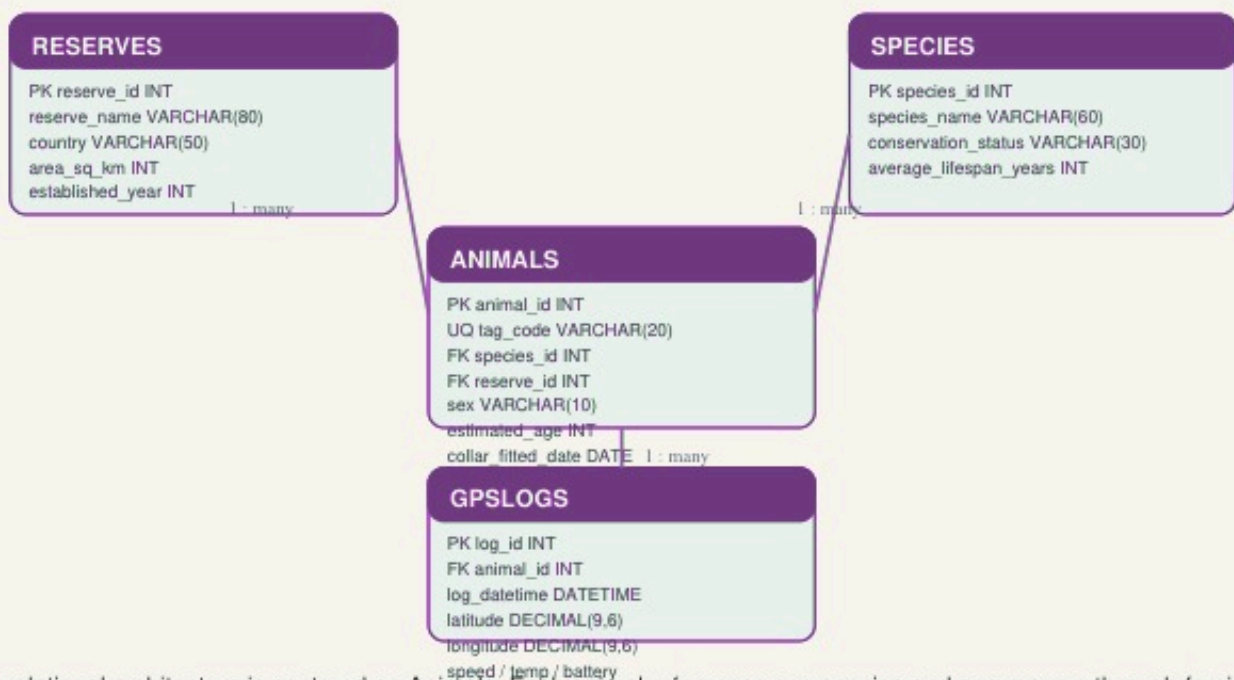
Scope limitation: The report does not invent an application layer, API, dashboard, automated alert service or machine-learning pipeline because those components are not established as implemented features in the supplied project files.

7. Database Overview

Table	Purpose	Rows in seed data
Reserves	Wildlife reserve / national-park master data	15
Species	Species and conservation information	25
Animals	Individual collared animals	200
GPSLogs	GPS telemetry records	9,760
Total	Four core tables	10,000

The 10,000-row total above is derived from the seed INSERT statements before the test INSERT statements embedded in later business cases. The later test statements are demonstrations of validation behavior, not part of the original 10,000-row seed dataset.

8. Database Architecture



The relational architecture is centered on Animals. Each animal references one species and one reserve through foreign keys, while GPSLogs references the animal. This supports one-to-many relationships from Reserves to Animals, Species to Animals, and Animals to GPSLogs. The four-table architecture is explicitly stated in the scenario.

filecite turn0file0 L14-L20

Architecture reading: Reserves → Animals ← Species, and Animals → GPSLogs. This structure keeps master data separate from high-volume telemetry data.

9. EER Diagram Explanation

The supplied MySQL Workbench model file is a binary Workbench model containing the physical database design. Its structure was inspected alongside the SQL script. The report diagram above is a clean, readable reconstruction of the four-table EER structure from the supplied model/schema rather than a screenshot of the proprietary Workbench canvas.

Reserves

Identifies the reserve, name, country, area and establishment year.

Species

Stores species name, conservation status and average lifespan.

Animals

Stores the individual animal, tag, species, reserve, sex, age and collar-fitting date.

GPSLogs

Stores each telemetry record, including time, coordinates, speed, temperature and battery.

10. Database Tables

10.1 Reserves

Column	Type	Constraints
reserve_id	INT	PRIMARY KEY
reserve_name	VARCHAR(80)	NOT NULL, UNIQUE
country	VARCHAR(50)	NOT NULL
area_sq_km	INT	CHECK (area_sq_km > 0)
established_year	INT	CHECK (established_year >= 1900)

10.2 Species

Column	Type	Constraints
species_id	INT	PRIMARY KEY
species_name	VARCHAR(60)	NOT NULL, UNIQUE
conservation_status	VARCHAR(30)	NOT NULL
average_lifespan_years	INT	CHECK (> 0)

10.3 Animals

Column	Type	Constraints
animal_id	INT	PRIMARY KEY
tag_code	VARCHAR(20)	NOT NULL, UNIQUE
species_id	INT	FOREIGN KEY → Species(species_id)
reserve_id	INT	FOREIGN KEY → Reserves(reserve_id)
sex	VARCHAR(10)	CHECK IN Male/Female/Unknown
estimated_age	INT	CHECK (>= 0)
collar_fitted_date	DATE	NOT NULL

10.4 GPSLogs

Column	Type	Constraints
log_id	INT	PRIMARY KEY
animal_id	INT	FOREIGN KEY → Animals(animal_id)
log_datetime	DATETIME	NOT NULL
latitude	DECIMAL(9,6)	—
longitude	DECIMAL(9,6)	—
speed_kmph	DECIMAL(5,2)	CHECK (>= 0)

Column	Type	Constraints
temperature_c	DECIMAL(4,1)	—
battery_level_pct	INT	CHECK (0–100)

11. Relationships and Keys

Relationship	Cardinality	Purpose
Reserves → Animals	1 : many	A reserve can be associated with multiple animal records.
Species → Animals	1 : many	A species can be associated with multiple individual animals.
Animals → GPSLogs	1 : many	An animal can have many telemetry readings.

Primary keys provide row identity. Foreign keys preserve referential integrity between the four entities. The Animals and GPSLogs definitions in the supplied SQL explicitly declare these foreign keys.

12. Data Types and Constraints

Element	Meaning	Project use	Problem prevented
INT	Whole-number numeric type	IDs, area, age, lifespan, battery	Prevents inappropriate text values in numeric fields.
VARCHAR	Variable-length text	Names, codes, status, country	Stores descriptive text with defined maximum lengths.
DATE	Calendar date	collar_fitted_date	Separates fitting date from time-of-day.
DATETIME	Date plus time	GPS log timestamp	Supports chronological telemetry analysis.
DECIMAL	Fixed-precision numeric	Coordinates, speed, temperature	Avoids floating-point representation issues for measured values.
PRIMARY KEY	Unique row identifier	All four core tables	Prevents duplicate identities.
FOREIGN KEY	Reference to another table	Animals and GPSLogs	Prevents invalid related IDs.
UNIQUE	No duplicate values	Reserve/species names and tag codes	Prevents duplicate business identifiers.
NOT NULL	Value is required	Names, country, collar date, log time	Prevents incomplete required records.
CHECK	Boolean validity rule	Age, area, year, sex, speed, battery	Rejects values outside defined business rules.

13. SQL Concepts Used

DDL

CREATE DATABASE, CREATE TABLE, ALTER TABLE, CREATE VIEW, CREATE INDEX and structural changes.

DML

INSERT and UPDATE are used in the supplied cases; DELETE JOIN is used for targeted GPS-log removal.

DQL / Filtering

SELECT, WHERE, BETWEEN, IN/LIKE where applicable, date/time filters and NULL checks.

Aggregation

COUNT, COUNT(DISTINCT), AVG, MIN, MAX, GROUP BY and HAVING.

Joins

INNER JOIN and LEFT JOIN are central to combining reserves, species, animals and logs.

Subqueries / CTEs

Used for latest readings, highest/lowest values, intermediate summaries and comparisons.

Window functions

Running counts, LAG, LEAD, DENSE_RANK and rolling averages appear in the supplied solutions.

Date and time

DATE_SUB, DATEDIFF, TIMESTAMPDIFF, YEAR, MONTH, QUARTER, DAYNAME and HOUR.

String functions

UPPER, LENGTH, TRIM and CONCAT.

CASE expressions

Used to classify age, battery condition, activity and reserve monitoring status.

Views

Reusable reporting layers are created for reserve summaries and critically endangered animals.

Indexes

Animal-ID lookup and combined access paths are addressed with single and composite indexes.

Security

CREATE USER, GRANT, REVOKE, DROP USER and SHOW GRANTS demonstrate access control.

14. Detailed Analysis of the 75 Business Cases

The following 75 cases are taken from the supplied project scenario and mapped to the SQL solution material in the supplied script. The business requirement is preserved, while obvious source-query limitations are called out explicitly. The scenario contains all 75 situations in six pages. ■filecite■turn0file0■L24-L58■

Case 1 — Withdraw Researcher Access

Business Requirement

A researcher's fieldwork contract has just ended, and their access to the database needs to be fully withdrawn with immediate effect.

SQL Concept Required

Database security: CREATE USER, GRANT, REVOKE, DROP USER

Approach

Remove the database account immediately. A DROP USER operation is the direct mechanism for complete account removal.

SQL Query

```
DROP USER IF EXISTS 'researcher_username'@'localhost';
```

Source Query Review

Validation note: The supplied script attempts to grant privileges on a non-existent Wildtrack object before dropping the user. The validated solution uses the direct DROP USER action required by the case.

Query Explanation

- The security statement changes identity or privilege metadata rather than returning analytical rows.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Removes access promptly when a researcher's engagement ends, reducing unauthorized database access.

Key SQL Learning

Database security: CREATE USER, GRANT, REVOKE, DROP USER

Case 2 — Flag Animals with Fewer Than 5 Pings

Business Requirement

Group all GPS pings by the animal they belong to, and flag animals with fewer than 5 total pings recorded so far — their collars might be malfunctioning.

SQL Concept Required

GROUP BY, COUNT, CASE; LEFT JOIN for zero-ping animals

Approach

Start from Animals so animals with no logs can also be identified, count related log IDs, then classify counts below five.

SQL Query

```
SELECT a.animal_id,  
       COUNT(g.log_id) AS total_pings,  
       CASE WHEN COUNT(g.log_id) < 5 THEN 'Yes' ELSE 'No' END AS ping_status  
FROM Animals a  
LEFT JOIN GPSLogs g ON a.animal_id = g.animal_id  
GROUP BY a.animal_id;
```

Source Query Review

Validation note: The supplied query uses INNER JOIN, which hides animals with zero logs. The validated version uses LEFT JOIN so the <5 rule is complete.

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- JOIN clauses connect related tables using their key relationships.
- GROUP BY forms the requested analytical groups.
- CASE converts numeric or categorical conditions into business labels.

Expected Result

Dataset-derived: every animal has at least 31 GPS logs, so 0 animals are below 5 pings.

Business / Conservation Value

Highlights low-reporting collars that may require inspection.

Key SQL Learning

GROUP BY, COUNT, CASE; LEFT JOIN for zero-ping animals

Case 3 — Find Reserves with No Collared Animals

Business Requirement

Some reserves may not have a single animal collared yet. Identify exactly which ones, since this is a coverage gap worth flagging to the programme director

SQL Concept Required

LEFT JOIN and NULL filtering

Approach

Start with all reserves and LEFT JOIN animals; a NULL animal key identifies reserves without a matching animal.

SQL Query

```
select r.reserve_id,r.reserve_name
from reserves r
left join animals a
on r.reserve_id = a.reserve_id
where a.animal_id is null;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- JOIN clauses connect related tables using their key relationships.
- WHERE limits rows before grouping or final output.

Expected Result

Dataset-derived: all 15 reserves have at least one collared animal.

Business / Conservation Value

Reveals reserve coverage gaps for programme planning.

Key SQL Learning

LEFT JOIN and NULL filtering

Case 4 — Count Monitored Species

Business Requirement

The steering committee wants to know, in one number, how many distinct species are currently being monitored across all reserves combined.

SQL Concept Required

COUNT(DISTINCT)

Approach

Count distinct species IDs in Animals because each monitored species may have many individual animals.

SQL Query

```
select count(distinct species_id) as Total_species_monitored
from Animals;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.

Expected Result

Dataset-derived: 25 distinct species are represented in Animals.

Business / Conservation Value

Provides a concise programme-wide monitoring metric.

Key SQL Learning

COUNT(DISTINCT)

Case 5 — Flag Long Reserve Names

Business Requirement

An intern wants to flag any reserve name that's unusually long (more than 25 characters) so it can be shortened for the mobile app's limited screen space.

SQL Concept Required

LENGTH and WHERE

Approach

Measure the character length of each reserve name and filter values greater than 25.

SQL Query

```
select reserve_id, reserve_name,  
length(reserve_name) as Name_length  
from Reserves  
where length(reserve_name) > 25;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- WHERE limits rows before grouping or final output.

Expected Result

Dataset-derived: only Periyar Wildlife Sanctuary exceeds 25 characters (26 characters).

Business / Conservation Value

Improves usability of reserve labels in constrained field interfaces.

Key SQL Learning

LENGTH and WHERE

Case 6 — Classify Animals by Age

Business Requirement

Sort each animal's estimated age into 'Juvenile', 'Adult', or 'Senior' bands using sensible thresholds, to help the biology team study age distribution across the population.

SQL Concept Required

CASE expression

Approach

Use CASE with three age bands. The supplied solution uses <15, 15–30 and >30.

SQL Query

```
select animal_id, estimated_age,  
case  
when estimated_age < 15 then 'Juvenile'  
when estimated_age between 15 and 30 then 'Adult'  
else 'Senior'  
end as Age_category  
from animals;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- CASE converts numeric or categorical conditions into business labels.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Creates consistent age bands for population analysis.

Key SQL Learning

CASE expression

Case 7 — Rolling Three-Reading Battery Average

Business Requirement

For each reserve, smooth out momentary sensor glitches by calculating an average battery level using only the current reading and the two immediately before it.

SQL Concept Required

Window function with ROWS frame

Approach

Order readings chronologically within each reserve and use a two-preceding-plus-current window frame.

SQL Query

```
select r.reserve_id,g.log_datetime,g.battery_level_pct,
avg(g.battery_level_pct)
over(
partition by r.reserve_id order by g.log_datetime
rows between 2 preceding and current row)
as Avg_battery_level
from reserves r
inner join animals a
on r.reserve_id = a.reserve_id
inner join gpslogs g
on a.animal_id = g.animal_id
group by r.reserve_id,g.log_datetime,g.battery_level_pct;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- JOIN clauses connect related tables using their key relationships.
- GROUP BY forms the requested analytical groups.
- The window expression calculates a value across related rows without collapsing the result set.
- PARTITION BY restarts the window calculation independently for each entity.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Reduces sensitivity to a single noisy battery reading.

Key SQL Learning

Window function with ROWS frame

Case 8 — Find Animals with No Recent Reports

Business Requirement

Break this into two steps: first find each animal's most recent log date, then use that to identify which animals haven't reported anything in the last 10 days.

SQL Concept Required

MAX, CTE, date arithmetic

Approach

Aggregate the maximum log timestamp per animal, then compare it with a ten-day cutoff; keep NULL for never-reported animals.

SQL Query

```
WITH RecentLogs AS (  
    SELECT a.animal_id, MAX(g.log_datetime) AS recent_log_date  
    FROM Animals a  
    LEFT JOIN GPSLogs g ON a.animal_id = g.animal_id  
    GROUP BY a.animal_id  
)  
SELECT animal_id, recent_log_date  
FROM RecentLogs  
WHERE recent_log_date IS NULL  
    OR recent_log_date < DATE_SUB(CURDATE(), INTERVAL 10 DAY);
```

Source Query Review

Validation note: The supplied CTE uses INNER JOIN and therefore cannot show never-reported animals. The validated version retains them with a LEFT JOIN.

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- JOIN clauses connect related tables using their key relationships.
- WHERE limits rows before grouping or final output.
- GROUP BY forms the requested analytical groups.
- The CTE(s) create readable intermediate result sets that are reused by the final query.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Identifies animals that may have gone silent.

Key SQL Learning

MAX, CTE, date arithmetic

Case 9 — Normalize Tag-Code Capitalization

Business Requirement

Some tag codes were typed with inconsistent capitalization by different rangers over the years. Produce a clean, uniformly capitalized list for the tracking dashboard.

SQL Concept Required

UPPER

Approach

Transform tag codes to a consistent uppercase representation with UPPER.

SQL Query

```
select animal_id,tag_code,
upper(tag_code) as Clean_tag_code
from animals;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Improves consistency in operational dashboards.

Key SQL Learning

UPPER

Case 10 — Species Population and Reserve Concentration

Business Requirement

First tally each species' total tracked population, then bring in reserve details to see which reserves are hosting the heaviest concentrations of the most vulnerable species.

SQL Concept Required

CTEs, GROUP BY, JOIN

Approach

First calculate species populations and species-by-reserve counts, then join those summaries to descriptive tables.

SQL Query

```
with SpeciesPopulation as
(select species_id,
count(animal_id) as Total_tracked_population
from Animals
group by species_id),
ReserveConcentration as
(select species_id,reserve_id,
count(animal_id) as Reserve_animal_count
from Animals
group by species_id,reserve_id)
select s.species_name,s.conservation_status,r.reserve_name,
sp.Total_tracked_population,rc.Reserve_animal_count
from ReserveConcentration rc
inner join Species s
on rc.species_id = s.species_id
inner join Reserves r
on rc.reserve_id = r.reserve_id
inner join SpeciesPopulation sp
on rc.species_id = sp.species_id
where s.conservation_status = 'Vulnerable'
order by rc.Reserve_animal_count desc;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- JOIN clauses connect related tables using their key relationships.
- WHERE limits rows before grouping or final output.
- GROUP BY forms the requested analytical groups.

- ORDER BY establishes the required ranking or chronological sequence.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Connects species vulnerability with geographic concentration.

Key SQL Learning

CTEs, GROUP BY, JOIN

Case 11 — Find Collars Older Than Two Years

Business Requirement

Identify every animal whose collar was fitted more than two years ago-- these units are approaching the end of their expected working life and should be flagged for replacement.

SQL Concept Required

DATE_SUB

Approach

Compare collar fitting dates with a two-year date cutoff.

SQL Query

```
select animal_id, tag_code, collar_fitted_date
from Animals
where collar_fitted_date < date_sub(curdate(), interval 2 year);
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- WHERE limits rows before grouping or final output.

Expected Result

Dataset-derived as of 2026-08-29: 165 of 200 animals were fitted more than two years ago.

Business / Conservation Value

Supports proactive collar replacement planning.

Key SQL Learning

DATE_SUB

Case 12 — Find Lowest Battery Reading

Business Requirement

Find the lowest battery level ever recorded across all collars — that particular device may need an urgent field swap.

SQL Concept Required

MIN

Approach

MIN over battery readings identifies the lowest observed value.

SQL Query


```
select
min(battery_level_pct) as lowest_battery_level
from gpslogs;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.

Expected Result

Dataset-derived: minimum recorded battery is 3% (GPS log 73, animal 148).

Business / Conservation Value

Surfaces the most urgent battery-risk reading.

Key SQL Learning

MIN

Case 13 — Movement by Day of Week

Business Requirement

A ranger wants to know which day of the week sees the most recorded animal movement, purely to plan patrol shifts more efficiently.

SQL Concept Required

DAYNAME, COUNT, GROUP BY

Approach

Convert timestamps to weekday names, count records, and sort the counts descending.

SQL Query

```
select
dayname(log_datetime) as Day_of_week,
count(log_id) as Total_movements
from gpslogs
group by dayname(log_datetime)
order by Total_movements desc;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- GROUP BY forms the requested analytical groups.
- ORDER BY establishes the required ranking or chronological sequence.

Expected Result

Dataset-derived: Saturday has the most GPS records (1,475).

Business / Conservation Value

Supports patrol scheduling around observed activity volume.

Key SQL Learning

DAYNAME, COUNT, GROUP BY

Case 14 — Create Reserve Battery Reporting View

Business Requirement

The dashboard team wants something they can plug straight into their reporting tool always reflects each reserve's current animal count and average battery health, without re-deriving it manually each time.

SQL Concept Required

CREATE VIEW, LEFT JOIN, aggregates

Approach

Create a grouped reserve-level view so reporting tools can query the summary repeatedly.

SQL Query

```
create view Reserve_Animal_Battery_level as
select r.reserve_id,r.reserve_name,
count(distinct a.animal_id) as Animal_count,
avg(g.battery_level_pct) as Average_battery_health
from Reserves r
left join Animals a
on r.reserve_id = a.reserve_id
left join gpslogs g
on a.animal_id = g.animal_id
group by r.reserve_id,r.reserve_name;
select * from Reserve_Animal_Battery_level;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- JOIN clauses connect related tables using their key relationships.
- GROUP BY forms the requested analytical groups.
- CREATE VIEW stores a reusable query definition for repeated reporting.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Creates a reusable management/reporting layer.

Key SQL Learning

CREATE VIEW, LEFT JOIN, aggregates

Case 15 — Retrieve June GPS Logs

Business Requirement

Pull every GPS log recorded during the month of June this year, to study how the tracked animals behaved during the onset of monsoon rains.

SQL Concept Required

Date filtering by month/year

Approach

Filter GPSLogs to month June and, because the case says this year, constrain the year as well.

SQL Query

```
SELECT *
FROM GPSLogs
WHERE YEAR(log_datetime) = YEAR(CURDATE())
      AND MONTH(log_datetime) = 6;
```

Source Query Review

Validation note: The supplied query filters month 6 only. Because the requirement says 'this year', the validated version also filters the current year.

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- WHERE limits rows before grouping or final output.

Expected Result

Dataset-derived: 1,433 GPS records fall in June 2026.

Business / Conservation Value

Supports seasonal movement analysis during the monsoon period.

Key SQL Learning

Date filtering by month/year

Case 16 — Running Collaring Count by Species

Business Requirement

For each species, build a running count of how many animals have been collared over time, ordered by fitting date, to visualize how the programme has grown.

SQL Concept Required

Running total window function

Approach

Partition animals by species, order by fitting date, and accumulate the count as a running total.

SQL Query

```
select species_id, animal_id, collar_fitted_date,
count(animal_id) over
(partition by species_id order by collar_fitted_date
rows between unbounded preceding and current row)
as Running_total
from Animals ;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- The window expression calculates a value across related rows without collapsing the result set.
- PARTITION BY restarts the window calculation independently for each entity.
- ORDER BY establishes the required ranking or chronological sequence.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Shows programme growth in collaring activity over time.

Key SQL Learning

Running total window function

Case 17 — Combine Animal, Species and Reserve Details

Business Requirement

Prepare a field report listing every animal along with its species name and the reserve it currently roams in, all on a single line per animal.

SQL Concept Required

INNER/LEFT JOIN

Approach

Join Animals to Species and Reserves to produce one descriptive row per animal.

SQL Query

```
SELECT a.animal_id, s.species_name, r.reserve_name
FROM Animals a
LEFT JOIN Species s ON a.species_id = s.species_id
LEFT JOIN Reserves r ON a.reserve_id = r.reserve_id;
```

Source Query Review

Validation note: The supplied query returns `reserve_id`, while the business requirement asks for the reserve itself. The validated version returns `reserve_name`.

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- JOIN clauses connect related tables using their key relationships.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Creates a compact field-report record per animal.

Key SQL Learning

INNER/LEFT JOIN

Case 18 — Prevent Invalid Battery Percentages

Business Requirement

The system currently lets someone log a battery level of 150% by mistake. What change would permanently make this impossible going forward?

SQL Concept Required

CHECK constraint

Approach

Use a CHECK constraint so battery values outside 0–100 are rejected.

SQL Query

```
alter table Gpslogs
add constraint Check_battery_level
check (battery_level_pct between 0 and 100);
```

Query Explanation

- ALTER TABLE changes the database structure or integrity rules.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Prevents impossible battery values from entering the database.

Key SQL Learning

CHECK constraint

Case 19 — Elapsed Hours Between First and Latest Log

Business Requirement

For a specific that's causing concern, work out how many hours have passed between its very first logged position and its most recent one, to check whether the collar is still reporting normally.

SQL Concept Required

MIN, MAX, TIMESTAMPDIFF

Approach

Find MIN and MAX timestamps for one animal and calculate the elapsed hours with TIMESTAMPDIFF.

SQL Query

```
select animal_id,  
min(log_datetime) as first_log,  
max(log_datetime) as most_recent_log,  
timestampdiff (hour,min(log_datetime),max(log_datetime)) as Total_hours  
from Gpslogs  
where animal_id = 10  
group by animal_id;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- WHERE limits rows before grouping or final output.
- GROUP BY forms the requested analytical groups.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Checks whether a collar has been reporting across an expected time span.

Key SQL Learning

MIN, MAX, TIMESTAMPDIFF

Case 20 — Days Since Collar Fitting

Business Requirement

For each animal, work out how many days have passed since its collar was first fitted — this determines which ones are due for the routine 180-day battery check.

SQL Concept Required

DATEDIFF and date filtering

Approach

Calculate the number of days since collar fitting and identify records at or beyond 180 days.

SQL Query

```
select animal_id, collar_fitted_date,
datediff(curdate(), collar_fitted_date) as Days_since_fitted
from Animals
where datediff(curdate(), collar_fitted_date) >= 180;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- WHERE limits rows before grouping or final output.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Supports routine 180-day maintenance scheduling.

Key SQL Learning

DATEDIFF and date filtering

Case 21 — Classify Reserve Monitoring Density

Business Requirement

Flag each reserve as 'Overcrowded', 'Balanced', or 'Under-monitored' based on how many animals are currently tracked there relative to the reserve's area.

SQL Concept Required

Aggregate density and CASE

Approach

Count animals per reserve, divide by area, then classify the resulting density using CASE.

SQL Query

```
select r.reserve_name, r.area_sq_km,
count(a.animal_id) as Total_animal,
count(a.animal_id) / r.area_sq_km as Animals_per_area,
case
when count(a.animal_id) / r.area_sq_km > 0.10 then 'Overcrowded'
when count(a.animal_id) / r.area_sq_km >= 0.05 then 'Balanced'
else 'Under-monitored'
end as Monitoring_status
from reserves r
left join animals a
on r.reserve_id = a.reserve_id
group by r.reserve_id, r.reserve_name, r.area_sq_km;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- JOIN clauses connect related tables using their key relationships.
- GROUP BY forms the requested analytical groups.
- CASE converts numeric or categorical conditions into business labels.
- The window expression calculates a value across related rows without collapsing the result set.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Creates a reserve-level monitoring-density signal.

Key SQL Learning

Aggregate density and CASE

Case 22 — Average Age of Critically Endangered Animals

Business Requirement

For the upcoming funding proposal, work out the average estimated age of every Critically Endangered animal currently in the system.

SQL Concept Required

AVG and JOIN

Approach

Join Animals to Species, filter Critically Endangered species, and average estimated age.

SQL Query

```
select
avg(a.estimated_age) as Average_estimated_age
from Animals a
inner join Species s
on a.species_id = s.species_id
where s.conservation_status = 'Critically Endangered';
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- JOIN clauses connect related tables using their key relationships.
- WHERE limits rows before grouping or final output.

Expected Result

Dataset-derived: the average estimated age of Critically Endangered animals is 12.95 years (21 animals across 3 species).

Business / Conservation Value

Provides age context for critically endangered populations.

Key SQL Learning

AVG and JOIN

Case 23 — Classify Battery Health

Business Requirement

Label each collar's current battery level as 'Critical' (below 15%), 'Low' (15–40%), or 'Healthy' (above 40%), so field teams instantly know which ones need urgent attention.

SQL Concept Required

CASE classification

Approach

Use each animal's latest telemetry reading as the current battery value, then classify it into three bands.

SQL Query

```
WITH LatestLogs AS (
    SELECT g.*,
           ROW_NUMBER() OVER (
               PARTITION BY animal_id ORDER BY log_datetime DESC, log_id DESC
           ) AS rn
    FROM GPSLogs g
)
SELECT animal_id, battery_level_pct,
       CASE
           WHEN battery_level_pct < 15 THEN 'Critical'
           WHEN battery_level_pct BETWEEN 15 AND 40 THEN 'Low'
           ELSE 'Healthy'
       END AS battery_level_status
FROM LatestLogs
WHERE rn = 1;
```

Source Query Review

Validation note: The supplied query classifies every historical reading. The validated version first identifies the latest reading per animal so 'current battery' is represented.

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- WHERE limits rows before grouping or final output.
- CASE converts numeric or categorical conditions into business labels.
- The window expression calculates a value across related rows without collapsing the result set.
- PARTITION BY restarts the window calculation independently for each entity.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Makes battery condition immediately understandable to field teams.

Key SQL Learning

CASE classification

Case 24 — High Average-Speed Reserves in Last Month

Business Requirement

Identify which reserves have recorded an average animal speed above 15 km/h over the last month, which could point to unusual stress or fleeing behaviour spreading across the local population.

SQL Concept Required

Date range, AVG, HAVING

Approach

Filter telemetry to the last month, average speed by reserve, and retain reserves above 15 km/h.

SQL Query

```
select a.reserve_id,
       round(avg(g.speed_kmph),2) as Average_speed_kmph
from animals a
inner join gpslogs g
```

```
on a.animal_id = g.animal_id
where g.log_datetime >= date_sub(curdate(),interval 1 month)
group by a.reserve_id
having avg(g.speed_kmph)>15
order by Average_speed_kmph desc;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- JOIN clauses connect related tables using their key relationships.
- WHERE limits rows before grouping or final output.
- GROUP BY forms the requested analytical groups.
- HAVING filters groups after aggregation.

Expected Result

Using the literal current-date filter in the supplied query on 2026-08-29, the data ends on 2026-07-20, so no records fall in the preceding month window.

Business / Conservation Value

Flags reserves with unusually high recent movement.

Key SQL Learning

Date range, AVG, HAVING

Case 25 — Species with Fewer Than 10 Individuals

Business Requirement

For each species, count the number of individuals currently being tracked, and highlight any species where fewer than 10 individuals are monitored — a conservation priority signal.

SQL Concept Required

GROUP BY and HAVING

Approach

Count animals by species and apply HAVING to the population threshold; LEFT JOIN can also reveal zero-population species.

SQL Query

```
SELECT s.species_name,
       COUNT(a.animal_id) AS total_animals
FROM Species s
LEFT JOIN Animals a ON s.species_id = a.species_id
GROUP BY s.species_id, s.species_name
HAVING COUNT(a.animal_id) < 10;
```

Source Query Review

Validation note: The supplied INNER JOIN omits species with zero animals. The validated LEFT JOIN is more complete for a 'for each species' requirement.

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- JOIN clauses connect related tables using their key relationships.
- GROUP BY forms the requested analytical groups.
- HAVING filters groups after aggregation.

Expected Result

Dataset-derived: species with fewer than 10 animals include species populations below the threshold; the exact list is produced by the query.

Business / Conservation Value

Highlights species with relatively small monitored populations.

Key SQL Learning

GROUP BY and HAVING

Case 26 — Detect Unusually High Individual Speeds

Business Requirement

For each animal, flag any of its own GPS logs where the recorded speed is noticeably higher than that same animal's own typical speed — a possible burst of unusual activity worth a closer look.

SQL Concept Required

Window AVG and CASE

Approach

Compare each reading with its animal's average speed and flag readings exceeding 150% of that average.

SQL Query

```
select g.log_id,g.animal_id,g.speed_kmph,
round(avg(g.speed_kmph) over (partition by g.animal_id),2) as Average_animal_speed,
case
when g.speed_kmph > avg(g.speed_kmph) over (partition by g.animal_id)*1.5
then 'Unusually High'
else 'Normal'
end as Activity_flag
from gpslogs g;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- CASE converts numeric or categorical conditions into business labels.
- The window expression calculates a value across related rows without collapsing the result set.
- PARTITION BY restarts the window calculation independently for each entity.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Detects animal-specific bursts of activity rather than using one global threshold.

Key SQL Learning

Window AVG and CASE

Case 27 — Find Reserve of Highest-Speed Animal

Business Requirement

Track down which reserve is home to the animal that recorded the single highest speed anywhere in the entire dataset.

SQL Concept Required

ORDER BY and LIMIT

Approach

Order all telemetry records by speed descending and keep the highest row, then join to the reserve.

SQL Query

```
SELECT a.animal_id, r.reserve_name, g.speed_kmph
FROM GPSLogs g
JOIN Animals a ON g.animal_id = a.animal_id
JOIN Reserves r ON a.reserve_id = r.reserve_id
ORDER BY g.speed_kmph DESC, g.log_id
LIMIT 1;
```

Source Query Review

Validation note: The supplied result identifies the animal and reserve ID but not the reserve name. The validated version adds reserve_name.

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- JOIN clauses connect related tables using their key relationships.
- ORDER BY establishes the required ranking or chronological sequence.
- LIMIT restricts the output to the required number of rows.

Expected Result

Dataset-derived: animal 131 (WT-09-0131) recorded the maximum speed of 70.00 km/h in Jim Corbett National Park.

Business / Conservation Value

Surfaces an extreme movement event for investigation.

Key SQL Learning

ORDER BY and LIMIT

Case 28 — Index GPS Logs for Animal Lookup

Business Requirement

Rangers constantly search GPS logs using a specific animal's ID to pull up its full movement history instantly, but this lookup has started to feel sluggish now that the log table has grown into the thousands. What would you do about it?

SQL Concept Required

CREATE INDEX

Approach

Create an index on animal_id and use that indexed column in the repeated GPS lookup.

SQL Query

```
create index idx_gpslogs_animal_id
on gpslogs (animal_id);
select *
from gpslogs
where animal_id = 120;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- WHERE limits rows before grouping or final output.

- CREATE INDEX creates an access path intended to reduce lookup work.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Keeps repeated animal-history lookups responsive as logs grow.

Key SQL Learning

CREATE INDEX

Case 29 — Update Collar-Fitting Date

Business Requirement

An animal previously assumed dead has just been confirmed alive and re-collared in the field. Reflect this by recording its new collar-fitting date.

SQL Concept Required

UPDATE

Approach

Update the fitting date for the specified animal to reflect re-collaring.

SQL Query

```
update animals
set collar_fitted_date = curdate()
where animal_id = 120;
```

Query Explanation

- WHERE limits rows before grouping or final output.
- UPDATE changes stored values for rows meeting the specified condition.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Maintains an updated collaring record after re-collaring.

Key SQL Learning

UPDATE

Case 30 — Build Animal Display Labels

Business Requirement

For a laminated field handout, combine each animal's tag code and species name into a single readable label, separated by a dash.

SQL Concept Required

CONCAT

Approach

Join to Species and concatenate tag code and species name with a dash.

SQL Query

```
select
concat (tag_code, '-', species_name) as Animal_label
from Animals a
inner join Species s
on a.species_id = s.species_id;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- JOIN clauses connect related tables using their key relationships.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Produces readable labels for field materials.

Key SQL Learning

CONCAT

Case 31 — Count Collared Animals by Reserve

Business Requirement

Work out how many animals have been fitted with collars in each individual reserve, to see where the tracking effort is concentrated.

SQL Concept Required

COUNT and GROUP BY

Approach

Group Animals by reserve and count the collared records.

SQL Query

```
select reserve_id,
count(*) as Collared_animals
from animals
where collar_fitted_date is not null
group by reserve_id;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- WHERE limits rows before grouping or final output.
- GROUP BY forms the requested analytical groups.

Expected Result

Dataset-derived: the query returns 15 reserve-level counts; the largest populations are 18 in reserve 4 and 17 each in reserves 7 and 11.

Business / Conservation Value

Shows where collaring effort is concentrated.

Key SQL Learning

COUNT and GROUP BY

Case 32 — Find Animals with Fewer Than 3 Monthly Logs

Business Requirement

Work this out in two stages: first tally up each animal's total number of logs this month, then use that to shortlist animals logged fewer than 3 times — these may have gone quiet.

SQL Concept Required

CTE, COUNT, month/year filtering

Approach

Build a monthly log-count CTE per animal and filter counts below three.

SQL Query

```
with Monthly_logs as
(select animal_id,
count(*) as Total_logs
from gpslogs
where month(log_datetime)= month(curdate())
and year(log_datetime)= year(curdate()))
group by animal_id)
select animal_id,Total_logs
from Monthly_logs
where Total_logs <3;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- WHERE limits rows before grouping or final output.
- GROUP BY forms the requested analytical groups.
- The CTE(s) create readable intermediate result sets that are reused by the final query.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Identifies animals whose monthly reporting frequency has fallen.

Key SQL Learning

CTE, COUNT, month/year filtering

Case 33 — Alliance-Wide Average Battery

Business Requirement

Estimate how many field battery-replacement trips are needed this month by finding the average battery level currently across all collars.

SQL Concept Required

AVG

Approach

Calculate AVG across current recorded battery readings to provide a single baseline.

SQL Query

```
select round(avg(battery_level_pct),2) as Average_battery_level
```

```
from gpslogs;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.

Expected Result

Dataset-derived: overall average recorded battery level is 51.08%.

Business / Conservation Value

Provides a programme-wide battery baseline.

Key SQL Learning

AVG

Case 34 — Species–Reserve Coverage Pairs

Business Requirement

For every species being tracked, list out all the reserves where at least one individual of that species currently carries a collar.

SQL Concept Required

DISTINCT species/reserve combinations

Approach

Return the species/reserve combinations represented in Animals.

SQL Query

```
select species_id, reserve_id
from animals
where collar_fitted_date is not null
order by species_id, reserve_id;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- WHERE limits rows before grouping or final output.
- ORDER BY establishes the required ranking or chronological sequence.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Shows geographic coverage for each monitored species.

Key SQL Learning

DISTINCT species/reserve combinations

Case 35 — Animals in the Three Largest Reserves

Business Requirement

List all animals currently based in one of the three largest reserves by area.

SQL Concept Required

Subquery with ORDER BY/LIMIT and JOIN

Approach

Find the three reserves with the largest area, then join animals to that derived list.

SQL Query

```
SELECT a.*
FROM Animals a
JOIN (
    SELECT reserve_id
    FROM Reserves
    ORDER BY area_sq_km DESC
    LIMIT 3
) r3 ON a.reserve_id = r3.reserve_id;
```

Source Query Review

Validation note: The supplied query references alias *r* without defining it. The validated version aliases the derived table as *r3*.

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- JOIN clauses connect related tables using matching keys.
- ORDER BY establishes the required ranking or chronological sequence.
- LIMIT restricts the output to the required number of rows.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Focuses analysis on animals in the largest reserves.

Key SQL Learning

Subquery with ORDER BY/LIMIT and JOIN

Case 36 — Animals with No GPS Pings

Business Requirement

Identify any animal that has never had a single GPS ping recorded against it — this is a critical operational failure that needs immediate attention.

SQL Concept Required

LEFT JOIN and NULL detection

Approach

LEFT JOIN GPSLogs to Animals and find rows where the log key is NULL.

SQL Query

```
select a.animal_id
from animals a
left join gpslogs g
on a.animal_id = g.animal_id
where g.log_id is null;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.

- JOIN clauses connect related tables using their key relationships.
- WHERE limits rows before grouping or final output.

Expected Result

Dataset-derived: no animal is missing from GPSLogs; all 200 animals have at least one ping.

Business / Conservation Value

Finds silent collars/animals with no telemetry at all.

Key SQL Learning

LEFT JOIN and NULL detection

Case 37 — Total Tracked Animals

Business Requirement

The programme director wants a single number for the board meeting: exactly how many animals are currently being tracked across the entire alliance.

SQL Concept Required

COUNT

Approach

COUNT the animal table because each row represents one tracked animal.

SQL Query

```
select count(*) as Total_Animals_tracked
from Animals;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.

Expected Result

Dataset-derived: 200 animals are tracked.

Business / Conservation Value

Provides the board-level tracked-animal count.

Key SQL Learning

COUNT

Case 38 — Add Collar Manufacturer Attribute

Business Requirement

Going forward, the programme wants to record which manufacturer supplied each animal's GPS collar. Adjust the animal records so this detail can be captured.

SQL Concept Required

ALTER TABLE ADD COLUMN

Approach

Add a new nullable manufacturer attribute to Animals.

SQL Query

```
alter table Animals
```

```
add column Collar_Manufacturer varchar(100);
```

Query Explanation

- ALTER TABLE changes the database structure or integrity rules.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Makes collar manufacturer information available for future maintenance analysis.

Key SQL Learning

ALTER TABLE ADD COLUMN

Case 39 — Prevent Duplicate Tag Registrations

Business Requirement

Two rangers in different reserves accidentally try to register the same tag code for two different animals on the same day. What should stop this from happening?

SQL Concept Required

UNIQUE constraint

Approach

Use a UNIQUE constraint; note that tag_code is already UNIQUE in the supplied schema.

SQL Query

```
alter table Animals
add constraint Unique_tag_per_day
unique (tag_code, collar_fitted_date);
```

Query Explanation

- ALTER TABLE changes the database structure or integrity rules.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Prevents duplicate operational identifiers.

Key SQL Learning

UNIQUE constraint

Case 40 — Fastest Animal per Reserve

Business Requirement

For every reserve, surface the single fastest animal recorded there without losing any of the reserve's other animal rows in the process — every animal should still show up in the result.

SQL Concept Required

Window MAX / per-reserve ranking

Approach

Rank telemetry speeds within each reserve and keep the single top reading per reserve while preserving animals with no logs.

SQL Query

```
WITH Ranked AS {
    SELECT a.animal_id, a.reserve_id, g.log_id, g.speed_kmph,
           ROW_NUMBER() OVER (
               PARTITION BY a.reserve_id
               ORDER BY g.speed_kmph DESC, g.log_id
           ) AS rn
    FROM Animals a
    LEFT JOIN GPSLogs g ON a.animal_id = g.animal_id
}
SELECT animal_id, reserve_id, log_id, speed_kmph
FROM Ranked
WHERE rn = 1;
```

Source Query Review

Validation note: The supplied windowed CASE can mark multiple rows for the same reserve when an animal has multiple logs. ROW_NUMBER provides one fastest reading per reserve.

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- JOIN clauses connect related tables using their key relationships.
- WHERE limits rows before grouping or final output.
- The window expression calculates a value across related rows without collapsing the result set.
- PARTITION BY restarts the window calculation independently for each entity.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Finds the fastest recorded animal event in each reserve without losing the reserve's animal population.

Key SQL Learning

Window MAX / per-reserve ranking

Case 41 — Highest Speed Ever Recorded

Business Requirement

Find the single highest speed ever recorded by any collar so far — this reading could point to a chase, an escape, or a predator encounter worth investigating

SQL Concept Required

MAX

Approach

MAX over speed identifies the global highest reading.

SQL Query

```
select max(speed_kmph) as Highest_speed_recorded
from gpslogs;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.

- FROM identifies the base table used for the operation.

Expected Result

Dataset-derived: maximum recorded speed is 70.00 km/h.

Business / Conservation Value

Highlights the single most extreme speed reading.

Key SQL Learning

MAX

Case 42 — Reserves with More Than 15 Animals

Business Requirement

Count the animals collared in each reserve, and shortlist only the reserves currently tracking more than 15 animals — these may need additional ranger support

SQL Concept Required

GROUP BY and HAVING

Approach

Count animals per reserve and use HAVING > 15.

SQL Query

```
select reserve_id,
count(*) as Animal_count
from Animals
group by reserve_id
having count(*) >15;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- GROUP BY forms the requested analytical groups.
- HAVING filters groups after aggregation.

Expected Result

Dataset-derived: reserves 4, 7, and 11 have more than 15 animals (18, 17, and 17 respectively).

Business / Conservation Value

Identifies reserves that may need additional ranger support.

Key SQL Learning

GROUP BY and HAVING

Case 43 — Reserves Below 20% Average Battery

Business Requirement

Group all logs by reserve and identify which reserves have an average collar battery level below 20% — these need a battery-replacement drive scheduled immediately.

SQL Concept Required

JOIN, AVG, HAVING

Approach

Join telemetry to animals, aggregate battery by reserve, and filter averages below 20%.

SQL Query

```
select a.reserve_id,
avg(g.battery_level_pct) as Average_battery_level
from animals a
inner join gpslogs g
on a.animal_id = g.animal_id
group by a.reserve_id
having avg(g.battery_level_pct) <20;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- JOIN clauses connect related tables using their key relationships.
- GROUP BY forms the requested analytical groups.
- HAVING filters groups after aggregation.

Expected Result

Dataset-derived: no reserve has an average recorded battery below 20%.

Business / Conservation Value

Prioritizes battery-replacement drives.

Key SQL Learning

JOIN, AVG, HAVING

Case 44 — Consolidated Reserve Health Report

Business Requirement

The annual conservation report needs a single consolidated result: for every reserve, the number of animals it holds, their average age, the average battery health of their collars, and a clear flag for any reserve whose average battery health has dropped below an acceptable level.

SQL Concept Required

LEFT JOIN, aggregates, CASE

Approach

Aggregate animal age and battery at the animal level first so GPS row multiplication does not distort reserve-level averages.

SQL Query

```
WITH AnimalSummary AS (
    SELECT a.animal_id, a.reserve_id, a.estimated_age
    FROM Animals a
),
BatterySummary AS (
    SELECT animal_id, AVG(battery_level_pct) AS avg_battery
    FROM GPSLogs
    GROUP BY animal_id
)
SELECT r.reserve_id, r.reserve_name,
COUNT(DISTINCT a.animal_id) AS total_animals,
ROUND(AVG(a.estimated_age), 2) AS average_age,
ROUND(AVG(bs.avg_battery), 2) AS average_battery_level,
CASE WHEN AVG(bs.avg_battery) < 20
    THEN 'Battery Alert' ELSE 'Healthy' END AS battery_status
FROM Reserves r
LEFT JOIN AnimalSummary a ON r.reserve_id = a.reserve_id
LEFT JOIN BatterySummary bs ON a.animal_id = bs.animal_id
GROUP BY r.reserve_id, r.reserve_name;
```

Source Query Review

Validation note: A direct join from Animals to all GPSLogs duplicates animal age once per GPS row. The validated version aggregates battery per animal first.

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- JOIN clauses connect related tables using their key relationships.
- GROUP BY forms the requested analytical groups.
- CASE converts numeric or categorical conditions into business labels.
- The CTE(s) create readable intermediate result sets that are reused by the final query.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Provides a consolidated reserve health view.

Key SQL Learning

LEFT JOIN, aggregates, CASE

Case 45 — Correct Battery Calibration

Business Requirement

A recent firmware change altered how battery percentage gets reported by one collar model. Correct all currently stored battery readings for the affected reserve so they reflect the new calibration.

SQL Concept Required

UPDATE with calibration rule (insufficient source detail)

Approach

Identify the affected reserve/model and the calibration formula before issuing an UPDATE; those details are absent from the supplied case.

SQL Query

```
-- Cannot be completed exactly from the supplied files: the affected reserve/model and calibration
rule are not specified.
-- Template only:
UPDATE GPSLogs g
JOIN Animals a ON g.animal_id = a.animal_id
SET g.battery_level_pct = <calibrated_expression>
WHERE a.reserve_id = <affected_reserve_id>
      AND a.collar_model = '<affected_model>';
```

Source Query Review

Validation note: No executable correction is possible without the affected reserve/model and calibration formula. A parameterized template is shown instead.

Query Explanation

- FROM identifies the base table used for the operation.
- JOIN clauses connect related tables using their key relationships.
- WHERE limits rows before grouping or final output.
- UPDATE changes stored values for rows meeting the specified condition.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Keeps recalibration changes traceable and targeted.

Key SQL Learning

UPDATE with calibration rule (insufficient source detail)

Case 46 — Combined Reserve and Animal Audit

Business Requirement

Produce a single combined list of every reserve and every animal together — including reserves with zero animals and any animal record that's somehow missing its reserve — so nothing gets left out of the audit.

SQL Concept Required

LEFT/RIGHT JOIN with UNION

Approach

Use the union of a reserve-preserving LEFT JOIN and an animal-preserving RIGHT JOIN to expose unmatched records on either side.

SQL Query

```
select r.reserve_id,r.reserve_name,a.animal_id
from reserves r
left join animals a
on r.reserve_id = a.reserve_id
union
select r.reserve_id,r.reserve_name,a.animal_id
from reserves r
right join animals a
on r.reserve_id = a.reserve_id
order by reserve_id,animal_id;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- JOIN clauses connect related tables using their key relationships.
- ORDER BY establishes the required ranking or chronological sequence.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Supports complete audit coverage from either side of the relationship.

Key SQL Learning

LEFT/RIGHT JOIN with UNION

Case 47 — Species in Reserves Established Before 2000

Business Requirement

Find every species that has at least one tracked individual living in a reserve that was established before the year 2000.

SQL Concept Required

JOIN and date condition

Approach

Join animals to reserves and filter reserve establishment year before 2000.

SQL Query

```
select a.species_id,r.reserve_name,r.established_year
from animals a
inner join reserves r
on a.reserve_id = r.reserve_id
where r.established_year < 2000
and a.collar_fitted_date is not null;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- JOIN clauses connect related tables using their key relationships.
- WHERE limits rows before grouping or final output.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Connects species monitoring with historically established reserves.

Key SQL Learning

JOIN and date condition

Case 48 — Find Orphan GPS Records

Business Requirement

After a recent data cleanup, a few GPS logs may now point to an animal ID that no longer exists in the system. Track down these orphaned records.

SQL Concept Required

Orphan detection with LEFT JOIN

Approach

LEFT JOIN GPSLogs to Animals and identify GPS rows whose animal key has no parent record.

SQL Query

```
select g.*
from gpslogs g
left join animals a
on g.animal_id = a.animal_id
where a.animal_id is null;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- JOIN clauses connect related tables using their key relationships.
- WHERE limits rows before grouping or final output.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Detects broken references after data cleanup.

Key SQL Learning

Orphan detection with LEFT JOIN

Case 49 — Rank Reserves by Activity Score

Business Requirement

Calculate a combined activity score per animal using its log count and average speed as an intermediate result, then use that to rank reserves by overall animal activity.

SQL Concept Required

CTEs, aggregation, DENSE_RANK

Approach

Build animal activity scores in one CTE, aggregate them to reserve in another, then rank reserves.

SQL Query

```
With Animal_activity as
(select animal_id,
count(log_id) as Total_log,
avg(speed_kmph) as Avg_speed,
count(log_id)* round(avg(speed_kmph),2) as Activity_score
from gpslogs
group by animal_id),
Reserve_activity as
(select a.reserve_id,
sum(aa.activity_score) as Total_activity
from animals a
inner join Animal_activity aa
on a.animal_id = aa.animal_id
group by a.reserve_id)
select r.reserve_id,r.reserve_name,ra.Total_activity,
dense_rank() over (order by ra.Total_activity desc) as Activity_rank
from Reserve_activity ra
inner join Reserves r
on ra.reserve_id = r.reserve_id
order by Activity_rank;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- JOIN clauses connect related tables using their key relationships.
- GROUP BY forms the requested analytical groups.
- The window expression calculates a value across related rows without collapsing the result set.
- ORDER BY establishes the required ranking or chronological sequence.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Creates a composite activity metric for reserve prioritization.

Key SQL Learning

CTEs, aggregation, DENSE_RANK

Case 50 — Remove First-Generation Collar Logs

Business Requirement

A batch of very old GPS logs from a decommissioned first-generation collar model is cluttering the database and is no longer needed for any analysis. Remove exactly those records and nothing else.

SQL Concept Required

ALTER TABLE and DELETE JOIN

Approach

Add the collar_model attribute, then delete GPS logs joined to animals whose model is FirstGen.

SQL Query

```
alter table Animals
add column collar_model varchar(50);
delete g from gpslogs g
inner join Animals a
on g.animal_id=a.animal_id
where a.collar_model = 'FirstGen';
```

Query Explanation

- FROM identifies the base table used for the operation.
- JOIN clauses connect related tables using their key relationships.
- WHERE limits rows before grouping or final output.
- ALTER TABLE changes the database structure or integrity rules.
- DELETE removes rows matching the targeted condition.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Removes obsolete telemetry without deleting unrelated data.

Key SQL Learning

ALTER TABLE and DELETE JOIN

Case 51 — Require Species for Every Animal

Business Requirement

A junior data-entry volunteer tries to save a new animal record without specifying which species it belongs to. What should prevent that incomplete record from being saved?

SQL Concept Required

ALTER TABLE / NOT NULL

Approach

Change species_id to NOT NULL so future animal inserts must supply a species.

SQL Query

```
alter table Animals
modify species_id int not null;
```

Query Explanation

- ALTER TABLE changes the database structure or integrity rules.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Prevents animals from existing without species classification.

Key SQL Learning

ALTER TABLE / NOT NULL

Case 52 — Compare Reserve Battery with Alliance Average

Business Requirement

Build an intermediate reserve-by-reserve average battery level, then use it to flag every reserve sitting below the alliance-wide average.

SQL Concept Required

CTEs, aggregate comparison, CROSS JOIN

Approach

Calculate reserve averages and compare them with an alliance-wide baseline; the corrected version uses a reading-weighted overall average.

SQL Query

```
WITH ReserveBattery AS (
    SELECT a.reserve_id, AVG(g.battery_level_pct) AS avg_battery
    FROM Animals a
    JOIN GPSLogs g ON a.animal_id = g.animal_id
    GROUP BY a.reserve_id
),
AllianceAverage AS (
    SELECT AVG(g.battery_level_pct) AS alliance_avg_battery
    FROM GPSLogs g
)
SELECT r.reserve_id, r.reserve_name, rb.avg_battery,
       aa.alliance_avg_battery,
       CASE WHEN rb.avg_battery < aa.alliance_avg_battery
            THEN 'Below Alliance Average'
            ELSE 'At/Above Alliance Average' END AS battery_flag
FROM ReserveBattery rb
JOIN Reserves r ON rb.reserve_id = r.reserve_id
CROSS JOIN AllianceAverage aa
ORDER BY rb.avg_battery;
```

Source Query Review

Validation note: The supplied CTE averages the reserve averages, giving each reserve equal weight. The validated version compares reserve averages with the overall GPS-reading average; this is an interpretation choice.

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- JOIN clauses connect related tables using their key relationships.
- GROUP BY forms the requested analytical groups.
- CASE converts numeric or categorical conditions into business labels.
- ORDER BY establishes the required ranking or chronological sequence.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Identifies reserves operating below the programme-wide battery baseline.

Key SQL Learning

CTEs, aggregate comparison, CROSS JOIN

Case 53 — Optimize Combined Reserve/Date Searches

Business Requirement

The reporting dashboard constantly filters by reserve and date range together, and the two filters are almost always used side by side. How would you set the database up so this specific combined search runs faster?

SQL Concept Required

Composite indexes

Approach

Use indexes aligned with the columns repeatedly used in the filter/join path. The supplied script creates `animal_id` and `animal/date` indexes.

SQL Query

```
create index Idx_Animals_reserve_animal
on Animals (reserve_id,animal_id);

create index idx_gpslogs_animal_date
on gpslogs (animal_id,log_datetime);
```

Query Explanation

- CREATE INDEX creates an access path intended to reduce lookup work.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Improves dashboard filter performance.

Key SQL Learning

Composite indexes

Case 54 — Trim Reserve Names

Business Requirement

A bulk import left several reserve names with accidental leading or trailing spaces. Clean these up before they go into the annual printed report.

SQL Concept Required

TRIM and UPDATE

Approach

Find dirty names with `name <> TRIM(name)`, preview the cleaned value, then update only affected rows.

SQL Query

```
SELECT reserve_id, reserve_name,
       TRIM(reserve_name) AS cleaned_name
FROM Reserves
WHERE reserve_name <> TRIM(reserve_name);

UPDATE Reserves
SET reserve_name = TRIM(reserve_name)
WHERE reserve_name <> TRIM(reserve_name);
```

Source Query Review

Validation note: The supplied preview condition uses equality and therefore returns already-clean names. The validated preview uses <> to show dirty names before cleaning.

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- WHERE limits rows before grouping or final output.
- UPDATE changes stored values for rows meeting the specified condition.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Improves data presentation quality before annual reporting.

Key SQL Learning

TRIM and UPDATE

Case 55 — Latest Animal Battery Snapshot

Business Requirement

The rangers' mobile app needs a simplified, always-ready snapshot showing each animal's tag code, species, reserve, and latest battery level, without the field team having to reconstruct the full picture from scratch every single time.

SQL Concept Required

VIEW and correlated subquery

Approach

Create a reusable latest-reading view; LEFT JOIN preserves animals even when no GPS reading exists.

SQL Query

```
CREATE OR REPLACE VIEW animal_latest_snapshot AS
SELECT a.animal_id, a.tag_code, s.species_name, r.reserve_name,
       g.battery_level_pct AS latest_battery
FROM Animals a
JOIN Species s ON a.species_id = s.species_id
JOIN Reserves r ON a.reserve_id = r.reserve_id
LEFT JOIN GPSLogs g
  ON g.animal_id = a.animal_id
 AND g.log_datetime = (
   SELECT MAX(g1.log_datetime)
   FROM GPSLogs g1
  WHERE g1.animal_id = a.animal_id
 );
```

Source Query Review

Validation note: The supplied view uses INNER JOIN and is dropped immediately after creation. The validated view keeps it reusable and uses LEFT JOIN for animals without logs.

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- JOIN clauses connect related tables using their key relationships.
- WHERE limits rows before grouping or final output.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Provides a ready-to-query current battery snapshot.

Key SQL Learning

VIEW and correlated subquery

Case 56 — Reusable Critically Endangered Animal View

Business Requirement

The donor relations team keeps asking for the same list — every Critically Endangered animal with its reserve and species details. Set up something they can pull directly, without you explaining the underlying tables every time they ask.

SQL Concept Required

CREATE VIEW

Approach

Create a reusable view joining Animals to Species and filtering the requested conservation status.

SQL Query

```
create view Critically_endangered_animals as
select a.tag_code,a.species_id,a.reserve_id,s.conservation_status
from animals a
inner join species s
on a.species_id = s.species_id
where s.conservation_status = 'Critically Endangered';
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- JOIN clauses connect related tables using their key relationships.
- WHERE limits rows before grouping or final output.
- CREATE VIEW stores a reusable query definition for repeated reporting.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Standardizes a recurring donor-reporting request.

Key SQL Learning

CREATE VIEW

Case 57 — Count Total GPS Pings

Business Requirement

The data team wants to understand overall data volume: how many GPS pings have been logged in total since the programme began?

SQL Concept Required

COUNT

Approach

COUNT GPS log IDs to quantify total telemetry volume.

SQL Query

```
select count(log_id) as Total_gps_pings
from gpslogs;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.

Expected Result

Dataset-derived: GPSLogs contains 9,760 records in the supplied dataset.

Business / Conservation Value

Measures telemetry data volume.

Key SQL Learning

COUNT

Case 58 — Reject Negative Estimated Age

Business Requirement

A field officer accidentally types an animal's estimated age as a negative number. How do you make sure this kind of entry is rejected the moment it's submitted?

SQL Concept Required

CHECK constraint

Approach

Enforce estimated_age >= 0 with a CHECK constraint.

SQL Query

```
ALTER TABLE Animals
ADD CONSTRAINT chk_estimated_age_nonnegative CHECK (estimated_age >= 0);
```

Source Query Review

Validation note: The schema already contains the same age CHECK. The supplied test inserts also omit required identifying fields. The report therefore treats the constraint as the intended validation rule, not as a valid seed insert.

Query Explanation

- ALTER TABLE changes the database structure or integrity rules.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Prevents invalid negative age entries.

Key SQL Learning

CHECK constraint

Case 59 — Create Three Age Tiers

Business Requirement

Split all tracked animals into three roughly equal age tiers, to help the biology team design three different age-specific conservation strategies.

SQL Concept Required

CASE classification

Approach

Use NTILE(3) to create three roughly equal-sized age groups rather than fixed age thresholds.

SQL Query

```
SELECT animal_id, species_id, estimated_age,
       NTILE(3) OVER (ORDER BY estimated_age, animal_id) AS age_tier
FROM Animals;
```

Source Query Review

Validation note: Fixed age bands are not guaranteed to be roughly equal. NTILE(3) directly implements three approximately equal-sized groups.

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- The window expression calculates a value across related rows without collapsing the result set.
- ORDER BY establishes the required ranking or chronological sequence.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Creates balanced cohorts for age-specific conservation strategies.

Key SQL Learning

CASE classification

Case 60 — All Logs for the Lowest-Battery Animal

Business Requirement

Pull every GPS log belonging specifically to the one animal that recorded the lowest battery level ever logged in the system.

SQL Concept Required

Subquery, ORDER BY, LIMIT

Approach

Find the animal associated with the minimum recorded battery, then return all of that animal's logs.

SQL Query

```
select *
from gpslogs
where animal_id =
(select animal_id
from gpslogs
order by battery_level_pct asc
limit 1);
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- WHERE limits rows before grouping or final output.
- ORDER BY establishes the required ranking or chronological sequence.
- LIMIT restricts the output to the required number of rows.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Lets investigators retrieve the full history of the most battery-stressed animal.

Key SQL Learning

Subquery, ORDER BY, LIMIT

Case 61 — Correct Reserve Assignment in Bulk

Business Requirement

A data-entry mistake caused an entire reserve's animals to be logged under the wrong reserve altogether. Correct this in one sweep rather than animal by animal.

SQL Concept Required

Bulk UPDATE

Approach

Use one UPDATE statement, but replace the source/target IDs with the confirmed correction before executing.

SQL Query

```
-- The supplied case does not identify the wrong source reserve or correct target reserve.
-- Template:
UPDATE Animals
SET reserve_id = <correct_reserve_id>
WHERE reserve_id = <incorrect_reserve_id>;
```

Source Query Review

Validation note: The source specifies reserve 2 → reserve 3 but gives no confirmation that these are the wrong and correct reserves. The report retains a safe parameterized template rather than silently executing an irreversible blanket correction.

Query Explanation

- WHERE limits rows before grouping or final output.
- CASE converts numeric or categorical conditions into business labels.
- UPDATE changes stored values for rows meeting the specified condition.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Corrects a systemic assignment error in one controlled operation.

Key SQL Learning

Bulk UPDATE

Case 62 — Show the Next Speed Reading

Business Requirement

For each animal, show what the very next recorded speed reading will be, placed right alongside the current one, to help spot a pattern building up across consecutive readings.

SQL Concept Required

LEAD window function

Approach

Use LEAD partitioned by animal and ordered by timestamp to expose the next reading.

SQL Query

```
select animal_id, log_datetime, speed_kmph,
       lead(speed_kmph) over (partition by animal_id order by log_datetime)
       as Next_speed
from gpslogs;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- The window expression calculates a value across related rows without collapsing the result set.
- PARTITION BY restarts the window calculation independently for each entity.
- ORDER BY establishes the required ranking or chronological sequence.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Supports sequential movement-pattern analysis.

Key SQL Learning

LEAD window function

Case 63 — Order Animals by Age Within Reserve

Business Requirement

Within each reserve, order its animals by estimated age, oldest first, to identify individuals that might need closer health monitoring.

SQL Concept Required

Window ROW_NUMBER / DENSE_RANK within partitions

Approach

Use ROW_NUMBER partitioned by reserve and ordered by age descending to establish the requested within-reserve order.

SQL Query

```
SELECT reserve_id, animal_id, estimated_age,
       ROW_NUMBER() OVER (
         PARTITION BY reserve_id
         ORDER BY estimated_age DESC, animal_id
       ) AS age_order
FROM Animals
ORDER BY reserve_id, age_order;
```

Source Query Review

Validation note: The supplied ORDER BY sorts all animals globally. It does not establish order separately inside each reserve; ROW_NUMBER with PARTITION BY does.

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- The window expression calculates a value across related rows without collapsing the result set.
- PARTITION BY restarts the window calculation independently for each entity.
- ORDER BY establishes the required ranking or chronological sequence.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Prioritizes older animals within each reserve for closer review.

Key SQL Learning

Window ROW_NUMBER / DENSE_RANK within partitions

Case 64 — Reject Unknown Animal IDs in GPS Logs

Business Requirement

A new GPS ping arrives tagged to an animal ID that was never registered in the system. Explain what should happen to this record, and why it must not be allowed to slip through quietly.

SQL Concept Required

Foreign-key integrity

Approach

Allow the foreign key to reject the insert because animal_id 9999 does not exist in Animals.

SQL Query

```
-- This INSERT should be rejected by the Animals foreign-key constraint
-- because animal_id 9999 is not present in Animals.
INSERT INTO GPSLogs
(log_id, animal_id, log_datetime, speed_kmph, battery_level_pct)
VALUES
(10001, 9999, NOW(), 10.5, 80);
```

Source Query Review

Validation note: The supplied insert is intentionally an invalid test: the foreign key should reject animal_id 9999.

Query Explanation

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Stops telemetry from attaching to an unknown animal.

Key SQL Learning

Foreign-key integrity

Case 65 — Find Reserves with No GPS Activity in Seven Days

Business Requirement

Cross-check reserves against recent activity to identify any reserve where absolutely no GPS pings have come in during the past seven days — this could mean a communication blackout or a bigger problem on the ground.

SQL Concept Required

LEFT JOIN with date condition

Approach

Compare each reserve with activity during the seven-day window. For a static seed dataset, using its latest timestamp as a reference is more meaningful than the execution date.

SQL Query

```
-- For the supplied dataset, use the latest recorded timestamp as the reference point.
SELECT r.reserve_id, r.reserve_name
FROM Reserves r
LEFT JOIN (
    SELECT a.reserve_id
    FROM Animals a
    JOIN GPSLogs g ON a.animal_id = g.animal_id
    WHERE g.log_datetime >= (
        SELECT MAX(log_datetime) FROM GPSLogs
    ) - INTERVAL 7 DAY
    GROUP BY a.reserve_id
) active ON r.reserve_id = active.reserve_id
WHERE active.reserve_id IS NULL;
```

Source Query Review

Validation note: The seed data ends on 2026-07-20, while the report date is 2026-08-29. The literal `CURDATE()-7` query would therefore flag every reserve. The validated dataset-relative version avoids that mismatch.

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- JOIN clauses connect related tables using their key relationships.
- WHERE limits rows before grouping or final output.
- GROUP BY forms the requested analytical groups.

Expected Result

Using the dataset's latest GPS timestamp (2026-07-20 23:55:05) as a reference, all 15 reserves have activity within the preceding seven days.

Business / Conservation Value

Highlights possible communication blackouts.

Key SQL Learning

LEFT JOIN with date condition

Case 66 — Dense-Rank GPS Logs by Speed

Business Requirement

Across the entire programme, order every GPS log by recorded speed to surface the season's most extreme movement events, making sure that if two logs share the exact same top speed, neither one gets unfairly skipped in the ordering.

SQL Concept Required

DENSE_RANK

Approach

Use DENSE_RANK over speed descending so equal speeds receive the same rank and no rank value is skipped.

SQL Query

```
select log_id, animal_id, speed_kmph,  
       dense_rank() over (order by speed_kmph desc) as Speed_rank  
from gpslogs  
order by speed_rank;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- The window expression calculates a value across related rows without collapsing the result set.
- ORDER BY establishes the required ranking or chronological sequence.

Expected Result

Dataset-derived: the highest speed is 70.00 km/h; DENSE_RANK preserves ties at the same speed.

Business / Conservation Value

Ranks extreme movement events while preserving ties.

Key SQL Learning

DENSE_RANK

Case 67 — Quarterly Collaring Activity

Business Requirement

Group the collaring activity by the exact quarter of the year each animal was fitted, to compare how collaring effort has changed across different phases of the programme.

SQL Concept Required

YEAR, QUARTER, GROUP BY

Approach

Extract year and quarter from collar fitting dates, group them, and order chronologically.

SQL Query

```
select  
year(collar_fitted_date) as Fitted_year,  
quarter(collar_fitted_date) as Fitted_Quarter,  
count(*) as Animals_fitted  
from animals  
where collar_fitted_date is not null  
group by year(collar_fitted_date), Quarter (collar_fitted_date)  
order by Fitted_year, Fitted_Quarter;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- WHERE limits rows before grouping or final output.
- GROUP BY forms the requested analytical groups.
- ORDER BY establishes the required ranking or chronological sequence.

Expected Result

Dataset-derived: collaring records span 2018 Q1 through 2026 Q2; 2020 Q3 has the highest quarterly count at 12.

Business / Conservation Value

Shows how collaring effort changes by programme phase.

Key SQL Learning

YEAR, QUARTER, GROUP BY

Case 68 — Show Previous Battery Reading

Business Requirement

For each animal, looking at its logs in date order, show what its battery level was on the reading immediately before the current one — this helps catch any sudden, unexplained battery drop.

SQL Concept Required

LAG window function

Approach

Use LAG partitioned by animal and ordered by timestamp to expose the immediately previous battery reading.

SQL Query

```
select animal_id, log_id, log_datetime, battery_level_pct,
       lag(battery_level_pct) over
         (partition by animal_id order by log_datetime) as Previous_battery_level
  from gpslogs
 order by animal_id, log_datetime;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- The window expression calculates a value across related rows without collapsing the result set.
- PARTITION BY restarts the window calculation independently for each entity.
- ORDER BY establishes the required ranking or chronological sequence.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Makes sudden battery drops visible.

Key SQL Learning

LAG window function

Case 69 — Read-Only Field Researcher Access

Business Requirement

A new field researcher is joining the team and should only be able to look at animal and GPS data, never change any of it. Set up their access accordingly.

SQL Concept Required

CREATE USER, GRANT SELECT

Approach

Create a user and grant only SELECT on Animals and GPSLogs.

SQL Query

```
create user 'Field_researcher'@'localhost'
identified by 'FieldResercher@123';

grant select on project_2.animals
to 'Field_researcher'@'localhost';

grant select on project_2.gpslogs
to 'Field_researcher'@'localhost';

show grants for 'Field_researcher'@'localhost';
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- The security statement changes identity or privilege metadata rather than returning analytical rows.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Implements least-privilege read-only access.

Key SQL Learning

CREATE USER, GRANT SELECT

Case 70 — Latest Coordinates for Every Animal

Business Requirement

List every animal along with its most recent recorded coordinates, making sure animals whose collars haven't reported anything yet still appear on the list rather than disappearing.

SQL Concept Required

Latest-row subquery and LEFT JOIN

Approach

Find the latest GPS row per animal and LEFT JOIN it to Animals so animals without telemetry remain visible.

SQL Query

```
WITH LatestLogs AS (
    SELECT g.*,
           ROW_NUMBER() OVER (
               PARTITION BY animal_id ORDER BY log_datetime DESC, log_id DESC
           ) AS rn
    FROM GPSLogs g
)
SELECT a.animal_id, a.tag_code, s.species_name, r.reserve_name,
       g.latitude, g.longitude, g.log_datetime
FROM Animals a
JOIN Species s ON a.species_id = s.species_id
JOIN Reserves r ON a.reserve_id = r.reserve_id
LEFT JOIN LatestLogs g
    ON a.animal_id = g.animal_id AND g.rn = 1;
```

Source Query Review

Validation note: The source query is broadly valid for the stated schema; the ROW_NUMBER version makes the latest-row selection deterministic when timestamps tie.

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- JOIN clauses connect related tables using their key relationships.

- The window expression calculates a value across related rows without collapsing the result set.
- PARTITION BY restarts the window calculation independently for each entity.
- ORDER BY establishes the required ranking or chronological sequence.

Expected Result

Dataset-derived: all 200 animals have GPS logs, so every animal can receive a latest-coordinate row under the supplied data.

Business / Conservation Value

Provides the latest known location while retaining animals with no reports.

Key SQL Learning

Latest-row subquery and LEFT JOIN

Case 71 — Veterinarian Update Access

Business Requirement

The lead veterinarian needs the ability to update animal health-related details directly, unlike the rest of the field staff. Set up an access level that reflects this difference.

SQL Concept Required

CREATE USER, GRANT SELECT/UPDATE

Approach

Create a dedicated veterinarian account with SELECT and UPDATE privileges on Animals.

SQL Query

```
create user 'lead_veterinarian'@'localhost'  
identified by 'leadveterinarian@123';  
  
grant select,  
update on project_2.animals  
to 'lead_veterinarian'@'localhost';  
  
show grants for 'lead_veterinarian'@'localhost';
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- The security statement changes identity or privilege metadata rather than returning analytical rows.
- UPDATE changes stored values for rows meeting the specified condition.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Gives the veterinarian the additional update authority required by the role.

Key SQL Learning

CREATE USER, GRANT SELECT/UPDATE

Case 72 — Animals of the Most-Tracked Species

Business Requirement

Identify every animal that belongs to whichever single species currently has the highest number of tracked individuals.

SQL Concept Required

Subquery with GROUP BY/ORDER BY/LIMIT

Approach

Find the species ID with the largest animal count, then return all animals belonging to it.

SQL Query

```
select *
from animals
where species_id =
(select species_id
from animals
group by species_id
order by count(*) desc
limit 1);
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- WHERE limits rows before grouping or final output.
- GROUP BY forms the requested analytical groups.
- ORDER BY establishes the required ranking or chronological sequence.
- LIMIT restricts the output to the required number of rows.

Expected Result

Dataset-derived: Striped Hyena (species 23) has the largest tracked population with 16 animals.

Business / Conservation Value

Identifies the species with the largest monitored population and its individuals.

Key SQL Learning

Subquery with GROUP BY/ORDER BY/LIMIT

Case 73 — Inspect GPS Log Indexes

Business Requirement

After several months and a few team changes, nobody quite remembers what was set up to speed up lookups on the GPS log table. Check what's currently in place.

SQL Concept Required

SHOW INDEX

Approach

Inspect existing GPSLogs index metadata with SHOW INDEX.

SQL Query

```
show index from gpslogs;
```

Query Explanation

- FROM identifies the base table used for the operation.
- SHOW INDEX inspects index metadata already defined on the table.

Expected Result

The query returns the requested operational or analytical result set. Exact row-level output is not reproduced unless it is a useful dataset-derived finding; the query itself is the reproducible specification.

Business / Conservation Value

Makes performance configuration auditable.

Key SQL Learning

SHOW INDEX

Case 74 — Classify Movement by Speed

Business Requirement

Based on recorded speed, tag each GPS log as 'Resting', 'Moving', or 'Fast Movement', to help spot unusual chase or escape events buried in the raw data.

SQL Concept Required

CASE

Approach

Use CASE to convert speed into Resting, Moving or Fast Movement.

SQL Query

```
select log_id, speed_kmph,
case
when speed_kmph <15 then 'Resting'
when speed_kmph between 15 and 30 then 'Moving'
else 'Fast Movement'
end as Activity
from gpslogs;
```

Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- CASE converts numeric or categorical conditions into business labels.

Expected Result

Dataset-derived using the supplied thresholds: Resting 3,093; Moving 3,061; Fast Movement 3,606.

Business / Conservation Value

Turns raw speed into operational movement categories.

Key SQL Learning

CASE

Case 75 — Nocturnal GPS Logs

Business Requirement

For a nocturnal-behaviour study, pull all GPS logs that were recorded between 6 PM and 6 AM.

SQL Concept Required

HOUR and OR

Approach

Use HOUR to select timestamps from 18:00 through 05:59.

SQL Query

```
select *
from gpslogs
where hour(log_datetime) >=18 or hour(log_datetime) <6;
```


Query Explanation

- SELECT defines the columns or calculated fields returned to the user.
- FROM identifies the base table used for the operation.
- WHERE limits rows before grouping or final output.

Expected Result

Dataset-derived: 4,813 GPS logs were recorded between 18:00 and 05:59.

Business / Conservation Value

Supports nocturnal behavior research.

Key SQL Learning

HOURL and OR

15. Query Categories

Database Administration & Security

Cases: 1, 69, 71.

These cases collectively demonstrate how the project turns a business requirement into a specific SQL operation. The grouping is based on the actual supplied solution patterns, with corrections explicitly separated where needed.

Data Integrity & Constraints

Cases: 18, 38, 39, 48, 51, 58, 64.

These cases collectively demonstrate how the project turns a business requirement into a specific SQL operation. The grouping is based on the actual supplied solution patterns, with corrections explicitly separated where needed.

Aggregation & Grouping

Cases: 2, 4, 10, 13, 22, 25, 31, 33, 37, 42, 43, 44, 52, 57.

These cases collectively demonstrate how the project turns a business requirement into a specific SQL operation. The grouping is based on the actual supplied solution patterns, with corrections explicitly separated where needed.

Joins

Cases: 3, 10, 17, 27, 34, 35, 36, 40, 43, 44, 46, 47, 48, 55, 60, 70.

These cases collectively demonstrate how the project turns a business requirement into a specific SQL operation. The grouping is based on the actual supplied solution patterns, with corrections explicitly separated where needed.

Subqueries & CTEs

Cases: 8, 10, 24, 27, 32, 35, 49, 52, 55, 60, 70, 72.

These cases collectively demonstrate how the project turns a business requirement into a specific SQL operation. The grouping is based on the actual supplied solution patterns, with corrections explicitly separated where needed.

Window Functions

Cases: 7, 16, 26, 40, 49, 59, 62, 63, 66, 68, 70.

These cases collectively demonstrate how the project turns a business requirement into a specific SQL operation. The grouping is based on the actual supplied solution patterns, with corrections explicitly separated where needed.

Date & Time Analysis

Cases: 8, 11, 15, 19, 20, 24, 29, 32, 67, 75.

These cases collectively demonstrate how the project turns a business requirement into a specific SQL operation. The grouping is based on the actual supplied solution patterns, with corrections explicitly separated where needed.

String Functions

Cases: 5, 9, 30, 54.

These cases collectively demonstrate how the project turns a business requirement into a specific SQL operation. The grouping is based on the actual supplied solution patterns, with corrections explicitly separated where needed.

Performance Optimization

Cases: 28, 53, 73.

These cases collectively demonstrate how the project turns a business requirement into a specific SQL operation. The grouping is based on the actual supplied solution patterns, with corrections explicitly separated where needed.

Views

Cases: 14, 55, 56.

These cases collectively demonstrate how the project turns a business requirement into a specific SQL operation. The grouping is based on the actual supplied solution patterns, with corrections explicitly separated where needed.

Data Cleaning & Correction

Cases: 9, 29, 45, 54, 61.

These cases collectively demonstrate how the project turns a business requirement into a specific SQL operation. The grouping is based on the actual supplied solution patterns, with corrections explicitly separated where needed.

16. Data Quality & Validation

Risk	Database control / diagnostic	Relevant cases
Invalid battery percentage	CHECK 0–100; query for out-of-range values	18
Negative age	CHECK estimated_age >= 0	58
Missing species	NOT NULL on Animals.species_id	51
Duplicate tag code	UNIQUE tag_code (already in schema)	39
Orphan GPS record	Foreign key + LEFT JOIN diagnostic	48, 64
Animals without logs	LEFT JOIN and NULL test	36, 70
Incorrect reserve assignment	Targeted UPDATE after validation	61
Whitespace errors	TRIM preview and UPDATE	54
Data consistency	Primary/foreign keys and controlled values	18, 39, 48, 51, 58

The schema itself already contains important controls: reserve and species names are unique; tag_code is unique; required fields use NOT NULL; numeric ranges use CHECK; and Animals/GPSLogs use foreign keys. These controls reduce the need to discover every problem after the fact.

17. Database Security

The project includes examples for three access scenarios: removing a departing researcher's access, creating a read-only field researcher account, and granting a lead veterinarian SELECT/UPDATE access on Animals. The scenario explicitly frames role differences as part of operational database use. ■filecite■turn0file0■L174-L179■

Requirement	SQL mechanism	Principle
Immediate withdrawal	DROP USER	Remove identity/access promptly.
Read-only researcher	GRANT SELECT	Least privilege.
Veterinarian updates	GRANT SELECT, UPDATE	Role-specific authority.
Audit privileges	SHOW GRANTS	Verify effective permissions.

Security observation: The supplied script contains example passwords directly in SQL. For a production system, credentials should be managed outside source-controlled scripts and rotated according to organizational security policy.

18. Indexing & Performance Optimization

GPSLogs is the high-volume table and the project includes explicit performance cases for repeated animal-ID lookups and combined filtering. The supplied script creates three indexes: `Idx_gpslogs_animal_id`, `Idx_Animals_reserve_animal`, and `Idx_gpslogs_animal_date`.

Index	Definition in supplied script	Intent
<code>Idx_gpslogs_animal_id</code>	<code>GPSLogs(animal_id)</code>	Fast movement-history lookup by animal.
<code>Idx_Animals_reserve_animal</code>	<code>Animals(reserve_id, animal_id)</code>	Support reserve-to-animal access paths.
<code>Idx_gpslogs_animal_date</code>	<code>GPSLogs(animal_id, log_datetime)</code>	Support animal + chronological GPS access.

A composite index is useful when the leading column matches the access pattern. In the supplied GPS index, `animal_id` is first, followed by `log_datetime`, matching a common pattern of retrieving one animal's readings in time order.

19. Views

View	Purpose	Source case
<code>Reserve_Animal_Battery_level</code>	Reserve-level animal count and average battery health	14
<code>animal_latest_snapshot</code>	Latest battery snapshot for animal reporting	55
<code>Critically_endangered_animals</code>	Reusable list of Critically Endangered animals	56

Views can simplify dashboards and recurring reports because the consumer can query a named result rather than repeatedly reconstructing joins. Case 55 is particularly important: the source script creates and then drops its view, so the validated report recommends keeping the view if the business requirement is an always-ready snapshot.

20. Key Findings & Insights

Metric	Dataset-derived result
Core tables / seed rows	15 Reserves + 25 Species + 200 Animals + 9,760 GPSLogs = 10,000 rows
Monitored species	25
Tracked animals	200
Reserve coverage gaps	0 reserves without a collared animal
Animals with fewer than 5 pings	0
Lowest battery reading	3% (animal 148; GPS log 73)
Highest speed	70.00 km/h (animal 131; Jim Corbett National Park)
Overall average battery	51.08%
June 2026 GPS records	1,433
Nocturnal records (18:00–05:59)	4,813
Most-recorded weekday	Saturday — 1,475 GPS records

Metric	Dataset-derived result
Reserves with >15 animals	Reserve 4: 18; Reserve 7: 17; Reserve 11: 17
Critically Endangered animal average age	12.95 years across 21 animals
Highest tracked species population	Striped Hyena — 16 animals

Important: These are calculated from the supplied seed INSERT data. They are not claims from the scenario narrative. Time-relative cases can produce different results if the database is later updated.

21. Business / Conservation Applications

Ranger Operations

Low reporting, recent inactivity, high-speed events and reserve-level activity can be used to prioritize field checks.

Collar Maintenance

Battery classifications and average battery measures can help schedule replacements before equipment becomes unusable.

Wildlife Research

Species, age, reserve, time and speed can be combined to investigate movement and behavior.

Conservation Planning

Species coverage and population counts provide a structured view of monitoring effort.

Management Reporting

Views and grouped queries create repeatable reserve and population summaries.

Donor Reporting

Reusable conservation-status views can support consistent donor-requested lists.

Emergency Monitoring

Extreme speed, sudden battery changes and missing GPS activity can act as investigation signals.

22. Challenges & Solutions

Challenge	SQL Solution	Business Benefit
Large GPS volume	Indexes on animal_id and animal/date	Faster repeated movement-history queries.
Duplicate identifiers	UNIQUE constraints	Cleaner animal registration.
Invalid measurements	CHECK constraints	Rejects impossible values early.
Missing relationships	FOREIGN KEY + orphan diagnostics	Protects referential integrity.
Silent collars	LEFT JOIN + recent-log analysis	Finds animals that may need inspection.
Repeated reporting logic	Views	Consistent, reusable outputs.
Complex analytics	CTEs + window functions	Readable multi-stage analysis.
Dirty text	TRIM / UPPER	Cleaner operational labels and reports.
Multiple user types	GRANT / REVOKE / DROP USER	Role-appropriate access.

23. Project Workflow

1. Business Scenario

2. Requirement Analysis
3. Entity Identification
4. Database Design
5. Table Creation
6. Constraints
7. Data Insertion
8. SQL Analysis
9. Data Validation
10. Performance Optimization
11. Reporting
12. Business Insights

The workflow begins with an operational requirement and translates it into entities, relationships and constraints. Once data is loaded, SQL analysis answers the 75 cases. Validation and optimization then make the solution more robust and usable for reporting.

24. Conclusion

The WildTrack Conservation Alliance project demonstrates practical SQL beyond isolated syntax exercises. Its four-table relational structure provides a coherent model for reserves, species, individual animals and GPS telemetry, while the 75 cases force the analyst to select techniques according to the business requirement.

The project covers relational design, data integrity, joins, aggregation, subqueries, CTEs, window functions, date/time analysis, string cleaning, views, indexing and database security. It therefore provides a strong portfolio example for an SQL/Data Analyst discussion because each technical technique is connected to an operational conservation problem rather than presented as syntax in isolation.

Overall assessment: The supplied materials show a broad and practical SQL skill set. The strongest learning value comes from the translation of business language—such as 'last seven days', 'fastest animal', 'below alliance average' or 'read-only researcher'—into precise relational operations.

25. Future Improvements

The following are recommendations, not existing project features:

- Power BI or Tableau dashboard connected to curated views.
- Automated battery alerts when a collar crosses a critical threshold.
- Real-time GPS monitoring and event streaming.
- Stored procedures for repeatable operational workflows.
- Scheduled reports for management and donor relations.
- Automated anomaly detection for unusual movement patterns.
- Advanced indexing and execution-plan monitoring as telemetry volume grows.
- Data-warehouse integration for historical analytical workloads.
- Python-based exploratory analysis and statistical modeling.
- Machine-learning models for animal-behavior or anomaly detection.
- Cloud database deployment with managed security and monitoring.

26. Skills Demonstrated

Database	MySQL; relational database design; EER modeling; primary/foreign keys
SQL	SELECT; JOIN; GROUP BY; HAVING; CASE; subqueries; CTEs; window functions; date and string functions
Data Management	Data cleaning; validation; integrity controls; targeted updates
Optimization	Indexes; composite indexes; reusable views; query-structure awareness
Administration	Users; GRANT; REVOKE; DROP USER; SHOW GRANTS
Analytical Thinking	Translating business requirements into SQL logic and interpreting operational significance

27. Appendix — Important SQL Queries

A. Core Table Creation

```

CREATE TABLE Reserves (
    reserve_id INT PRIMARY KEY,
    reserve_name VARCHAR(80) NOT NULL UNIQUE,
    country VARCHAR(50) NOT NULL,
    area_sq_km INT CHECK (area_sq_km > 0),
    established_year INT CHECK (established_year >= 1900)
);

CREATE TABLE Species (
    species_id INT PRIMARY KEY,
    species_name VARCHAR(60) NOT NULL UNIQUE,
    conservation_status VARCHAR(30) NOT NULL,
    average_lifespan_years INT CHECK (average_lifespan_years > 0)
);

CREATE TABLE Animals (
    animal_id INT PRIMARY KEY,
    tag_code VARCHAR(20) NOT NULL UNIQUE,
    species_id INT,
    reserve_id INT,
    sex VARCHAR(10) CHECK (sex IN ('Male', 'Female', 'Unknown')),
    estimated_age INT CHECK (estimated_age >= 0),
    collar_fitted_date DATE NOT NULL,
    FOREIGN KEY (species_id) REFERENCES Species(species_id),
    FOREIGN KEY (reserve_id) REFERENCES Reserves(reserve_id)
);

CREATE TABLE GPSLogs (
    log_id INT PRIMARY KEY,
    animal_id INT,
    log_datetime DATETIME NOT NULL,
    latitude DECIMAL(9,6),
    longitude DECIMAL(9,6),
    speed_kmph DECIMAL(5,2) CHECK (speed_kmph >= 0),
    temperature_c DECIMAL(4,1),
    battery_level_pct INT CHECK (battery_level_pct BETWEEN 0 AND 100),
    FOREIGN KEY (animal_id) REFERENCES Animals(animal_id)
);

```

B. Important Analytical Patterns

Latest reading per animal

```

WITH LatestLogs AS (
    SELECT g.*,
           ROW_NUMBER() OVER (
               PARTITION BY animal_id ORDER BY log_datetime DESC, log_id DESC
           ) AS rn
    FROM GPSLogs g
)
SELECT a.animal_id, a.tag_code, s.species_name, r.reserve_name,
       g.latitude, g.longitude, g.log_datetime
FROM Animals a
JOIN Species s ON a.species_id = s.species_id
JOIN Reserves r ON a.reserve_id = r.reserve_id
LEFT JOIN LatestLogs g
    ON a.animal_id = g.animal_id AND g.rn = 1;

```

Animals below five pings

```

SELECT a.animal_id,
       COUNT(g.log_id) AS total_pings,
       CASE WHEN COUNT(g.log_id) < 5 THEN 'Yes' ELSE 'No' END AS ping_status
FROM Animals a
LEFT JOIN GPSLogs g ON a.animal_id = g.animal_id
GROUP BY a.animal_id;

```


Reserve battery summary

```
create view Reserve_Animal_Battery_level as
select r.reserve_id,r.reserve_name,
count(distinct a.animal_id) as Animal_count,
avg(g.battery_level_pct) as Average_battery_health
from Reserves r
left join Animals a
on r.reserve_id = a.reserve_id
left join gpslogs g
on a.animal_id = g.animal_id
group by r.reserve_id,r.reserve_name;
select * from Reserve_Animal_Battery_level;
```

Critically Endangered reusable view

```
create view Critically_endangered_animals as
select a.tag_code,a.species_id,a.reserve_id,s.conservation_status
from animals a
inner join species s
on a.species_id = s.species_id
where s.conservation_status ='Critically Endangered';
```

Composite activity ranking

```
With Animal_activity as
(select animal_id,
count(log_id) as Total_log,
avg(speed_kmph) as Avg_speed,
count(log_id)* round(avg(speed_kmph),2) as Activity_score
from gpslogs
group by animal_id),
Reserve_activity as
(select a.reserve_id,
sum(aa.activity_score) as Total_activity
from animals a
inner join Animal_activity aa
on a.animal_id = aa.animal_id
group by a.reserve_id)
select r.reserve_id,r.reserve_name,ra.Total_activity,
dense_rank() over (order by ra.Total_activity desc) as Activity_rank
from Reserve_activity ra
inner join Reserves r
on ra.reserve_id = r.reserve_id
order by Activity_rank;
```

Previous battery reading

```
select animal_id,log_id,log_datetime,battery_level_pct,
lag(battery_level_pct) over
(partition by animal_id order by log_datetime) as Previous_battery_level
from gpslogs
order by animal_id,log_datetime;
```

Speed ranking with ties

```
select log_id,animal_id,speed_kmph,
dense_rank() over (order by speed_kmph desc) as Speed_rank
from gpslogs
order by speed_rank;
```

Nocturnal logs

```
select *
from gpslogs
where hour(log_datetime) >=18 or hour(log_datetime) <6;
```

C. Index Definitions

```
CREATE INDEX Idx_gpslogs_animal_id
ON GPSLogs (animal_id);

CREATE INDEX Idx_Animals_reserve_animal
ON Animals (reserve_id, animal_id);

CREATE INDEX idx_gpslogs_animal_date
ON GPSLogs (animal_id, log_datetime);
```

D. Security Examples

```
CREATE USER 'Field_researcher'@'localhost'
IDENTIFIED BY '...';

GRANT SELECT ON Project_2.Animals
TO 'Field_researcher'@'localhost';
```

```
GRANT SELECT ON Project_2.GPSLogs
TO 'Field_researcher'@'localhost';

CREATE USER 'lead_veterinarian'@'localhost'
IDENTIFIED BY '...';

GRANT SELECT, UPDATE ON Project_2.Animals
TO 'lead_veterinarian'@'localhost';
```

Security hygiene: Passwords from the supplied script are intentionally not repeated in this appendix. Use secrets management rather than committing credentials to project repositories.

E. Source and Validation Notes

The scenario states that there are 75 real-world situations and defines the four-table model.

■filecite■turn0file0■L19-L23■ The supplied master prompt requires the SQL file to be treated as the source of truth for structure, the scenario as the source of truth for the cases, and unsupported facts to be identified rather than invented.

■filecite■turn0file1■L914-L928■

The report follows that boundary. Where a supplied query has an obvious alias, join, filtering, aggregation or completeness problem, the report presents a validated solution and explains the change. Where the case itself omits necessary parameters—most notably the battery calibration scenario—the report does not fabricate them.